

PROMPT TO PROFIT™

WORKBOOK 02

Customer Record Management System

Build a Complete Customer Management Application with AI

TECHNOLOGY STACK

HTML

CSS

Vanilla JavaScript

Supabase

DIFFICULTY

Beginner

ESTIMATED COMPLETION TIME

25-30 Hours

PRODUCED BY

Tochukwu Tech and AI Academy

www.tochukwunkwocha.com

PROMPT TO PROFIT™

Workbook 02 · Customer Record Management System

© 2026 Tochukwu Tech and AI Academy. All rights reserved.

Produced and published by Tochukwu Tech and AI Academy.

www.tochukwunkwocha.com

This workbook is designed for personal learning and guided software-building practice.

Technology names and product names remain the property of their respective owners.

Contents

Workbook 02 · Customer Record Management System

LEARNER SUPPORT TOOLKIT	7
VISUAL GLOSSARY	8
WORK SAFELY: MAKE A BACKUP	10
SOMETHING WENT WRONG – WHAT SHOULD I CHECK?	11
MY ERROR LOG	13
CHAPTER 1 · BUILDING THE PUBLIC WEBSITE	15
LESSON 1 · UNDERSTANDING THE CUSTOMER RECORD MANAGEMENT SYSTEM	17
LESSON 2 · BUILDING THE LANDING PAGE	21
LESSON 3 · STYLING THE LANDING PAGE	35
LESSON 4 · ADDING THE RESPONSIVE NAVIGATION	46
LESSON 5 · COMPLETING THE PUBLIC WEBSITE	55
CHAPTER SUMMARY	64
CHAPTER MILESTONE	65
CHAPTER 2 · CONNECTING TO SUPABASE	67
LESSON 1 · CREATING YOUR SUPABASE PROJECT	69
LESSON 2 · UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY	74
LESSON 3 · CREATING YOUR SUPABASE CONNECTION FILE	79
LESSON 4 · BUILDING A CONNECTION TEST PAGE	86
LESSON 5 · FINAL CONNECTION REVIEW	94
CHAPTER SUMMARY	99
CHAPTER MILESTONE	100
CHAPTER 3 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	101
LESSON 1 · UNDERSTANDING THE AUTHENTICATION FLOW	104
LESSON 2 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	108
LESSON 3 · AUDITING THE AUTHENTICATION SYSTEM	130

CHAPTER SUMMARY	135
CHAPTER MILESTONE	136
CHAPTER 4 · BUILDING THE CUSTOMER DATABASE	137
LESSON 1 · UNDERSTANDING THE CUSTOMER DATA MODEL	139
LESSON 2 · CREATING THE CUSTOMERS TABLE	144
LESSON 3 · IMPROVING DATA QUALITY WITH DATABASE CONSTRAINTS	152
LESSON 4 · ENABLING ROW LEVEL SECURITY	157
LESSON 5 · CREATING THE SELECT POLICY	160
LESSON 6 · CREATING THE INSERT POLICY	164
LESSON 7 · CREATING THE UPDATE POLICY	168
LESSON 8 · CREATING THE DELETE POLICY	173
LESSON 9 · TESTING THE CUSTOMER DATABASE	177
CHAPTER SUMMARY	182
CHAPTER MILESTONE	183
CHAPTER 5 · BUILDING THE CUSTOMER DASHBOARD	184
LESSON 1 · DESIGNING THE CUSTOMER DASHBOARD	186
LESSON 2 · BUILDING THE CUSTOMER ENTRY FORM	195
LESSON 3 · SAVING CUSTOMER RECORDS SECURELY	206
LESSON 4 · BUILDING BUSINESS DASHBOARD STATISTICS	215
CHAPTER SUMMARY	223
CHAPTER MILESTONE	224
CHAPTER 6 · VIEWING CUSTOMERS	225
LESSON 1 · BUILDING THE CUSTOMER DIRECTORY	227
LESSON 2 · BUILDING THE CUSTOMER PROFILE	233
LESSON 3 · TESTING THE CUSTOMER DIRECTORY	244
CHAPTER SUMMARY	250
CHAPTER MILESTONE	251
CHAPTER 7 · SEARCHING, FILTERING AND SORTING CUSTOMERS	252
LESSON 1 · BUILDING CUSTOMER SEARCH, FILTERING AND SORTING	254
LESSON 2 · TESTING CUSTOMER DISCOVERY	264
CHAPTER SUMMARY	268

CHAPTER MILESTONE	269
CHAPTER 8 · EDITING CUSTOMER RECORDS	270
LESSON 1 · BUILDING CUSTOMER EDITING	272
LESSON 2 · TESTING CUSTOMER EDITING	282
CHAPTER SUMMARY	286
CHAPTER MILESTONE	287
CHAPTER 9 · DELETING CUSTOMER RECORDS	288
LESSON 1 · BUILDING SECURE CUSTOMER DELETION	290
LESSON 2 · TESTING CUSTOMER DELETION	298
CHAPTER SUMMARY	302
CHAPTER MILESTONE	303
CHAPTER 10 · FINAL APPLICATION REVIEW	304
LESSON 1 · COMPLETE SYSTEM TESTING	306
REFLECTION QUESTIONS	315
EXTENSION CHALLENGES	316
NEXT WORKBOOK	317

ABOUT THIS WORKBOOK

This workbook is a complete, self-contained project.

You do not need to own any of the other Prompt to Profit™ Software Workbooks to complete it successfully.

Every workbook in this series teaches you how to build one complete software application from start to finish. It contains all the explanations, prompts, instructions and practical exercises you need to complete that project independently.

Although some software development concepts naturally appear across multiple workbooks, every workbook starts from the beginning of its own project and assumes no prior knowledge of the other workbooks.

This means you can study the workbooks in any order that suits your goals.

If your immediate need is to build a Quotation Generator, you can start with that workbook.

If you need a Customer Record Management System, you can begin with this workbook.

If you later decide to build additional software, each new workbook will teach you a different business application while reinforcing the software development skills you have already learned.

Together, the workbooks form a complete software development series. Individually, each workbook is a complete learning experience.

WELCOME

Welcome to Prompt to Profit™ Workbook 02.

In this workbook, you will build a complete Customer Record Management System from the ground up using HTML, CSS, Vanilla JavaScript, Supabase and Artificial Intelligence.

Throughout this project, you will learn how to build secure, database-driven business software by following the same step-by-step development process used in real-world software projects.

You will create a professional customer management application that allows businesses to register users, manage customer information securely, search and organise customer records, update customer details, and protect every user's data using Row Level Security.

Every feature is built one step at a time. You won't simply copy prompts into an AI tool. You'll learn why each feature is being built, how the different parts of the application work together, and how to communicate effectively with AI to build reliable software.

By the end of this workbook, you will have a fully functional Customer Record Management System that you can proudly include in your software development portfolio.

You will build the project using:

- HTML
- CSS
- Vanilla JavaScript
- Supabase
- ChatGPT
- Notepad
- A web browser

You do not need to install additional software.

You will continue working with complete files and clear Build Prompts.

Every new feature will preserve everything already built.

Complete the lessons in order.

Test each feature before moving forward.

Do not continue when an important feature is not working.

WHAT YOU WILL BUILD

By the end of this workbook, your Customer Record Management System will include:

PUBLIC WEBSITE

- Responsive navigation
- Three-line hamburger menu
- Hero section
- Business problem section
- Features section
- How It Works section
- Customer dashboard preview
- Final call-to-action
- Footer
- Login and Register links

SUPABASE

- Supabase project
- Shared connection file
- Publishable Key
- Connection test page
- Correct script loading order

AUTHENTICATION

- User registration
- Email verification
- User login
- Protected dashboard
- Protected customer profile page
- Logout
- Friendly messages
- Loading states
- Disabled button states

CUSTOMER DATABASE

- Secure customers table
- User-owned customer records
- Required and optional fields
- Customer type
- Contact information
- Company information
- Address information
- Notes
- Created date
- Updated date

- Row Level Security

CUSTOMER MANAGEMENT

- Add customers
- View customer cards
- Open customer profiles
- Edit customers
- Delete customers
- Duplicate email detection
- Duplicate phone detection
- Click-to-call
- Click-to-email
- Search
- Filters
- Sorting

BUSINESS DASHBOARD

- Total Customers
- Customers Added Today
- New Customers This Month
- Business Customers
- Loading states
- Empty states
- Error states
- Responsive layout

COMPLETION

- Complete application audit
- Two-account privacy testing
- Final folder review

- Netlify deployment
- Live application testing
- Portfolio description
- Reflection questions
- Extension challenges

WORKBOOK STRUCTURE

This workbook is organised into ten chapters.

Chapter 1

Building the Public Website

Chapter 2

Connecting to Supabase

Chapter 3

Building the Complete Authentication System

Chapter 4

Building the Customer Database

Chapter 5

Building the Customer Dashboard

Chapter 6

Viewing Customers

Chapter 7

Building the Customer Profile Page

Chapter 8

Editing and Deleting Customer Records

Chapter 9

Making Customer Management More Powerful

Chapter 10

Final Testing and Project Completion

Complete each chapter before moving to the next.

Every chapter depends on the work completed earlier.

LEARNER SUPPORT TOOLKIT

Keep this part of the workbook close while you build.

It explains common words, shows you how to protect your work, gives you a simple way to investigate problems, and provides a place to record errors and solutions.

You are not expected to remember everything immediately.

Return to these pages whenever you need them.

VISUAL GLOSSARY

HTML	The file that gives a web page its content and structure.
CSS	The file that controls colours, spacing, layout and appearance.
VANILLA JAVASCRIPT	JavaScript used without a framework. It controls what the application does.
BROWSER	The program used to open and test the application, such as Chrome or Edge.
FILE	One saved part of the project, such as index.html or dashboard.js.
FOLDER	The place where all files for one project are kept together.
FUNCTION	A named set of instructions that performs one task.
EVENT	An action the browser can notice, such as a click, form submission or key press.
VARIABLE	A named place used to hold information while the application is running.
URL	The address of a page, website or online service.
SUPABASE	The online service used for the database, authentication and security.
DATABASE	An organised place where the application stores information.
TABLE	A named part of the database that stores one type of information.
RECORD	One complete item saved inside a database table.
AUTHENTICATION	The process of creating accounts, signing in and identifying the current user.
SESSION	The saved sign-in state that allows the application to remember the current user.
ROW LEVEL SECURITY	Supabase rules that control which database records each user can access.

PUBLISHABLE KEY	The Supabase project key that browser-based applications are allowed to use.
DEPLOY	To place the complete project online so it has a live HTTPS website address.
DEBUGGING	The careful process of finding, understanding and correcting a problem.

WORK SAFELY: MAKE A BACKUP

A backup is a separate copy of your complete project folder.

Create a backup before replacing files that already contain working code.

1.

Close any project files that are open in Notepad.

2.

Find your complete project folder.

3.

Copy the folder.

4.

Paste the copy outside the working project folder.

Do not place the backup inside the folder you deploy.

5.

Rename the copy clearly.

For example:

Project Backup - Before Chapter 3 Lesson 2

6.

Open the backup folder.

Confirm that the expected files are inside it.

If a new change causes a serious problem, you can return to this working copy.

SOMETHING WENT WRONG — WHAT SHOULD I CHECK?

Problems are a normal part of building software.

Do not replace random pieces of code and do not continue to the next lesson.

Work through this checklist in order.

- ☐ Confirm that you opened the correct project folder.
- ☐ Confirm that every updated file was saved in Notepad.
- ☐ Check every filename carefully.
- ☐ Make sure an HTML, CSS or JavaScript file does not end with .txt.
- ☐ Confirm that you pasted the complete file returned by AI.
- ☐ Confirm that you did not paste an explanation, a code-block label or only part of a file.
- ☐ Check that stylesheet and JavaScript filenames match the names used inside the HTML.
- ☐ On Supabase pages, check that the scripts load in the order taught in the lesson.
- ☐ Read the exact message shown on the page.
- ☐ Open the browser Console and look for the first red error.

To open the Console:

Right-click the page.

Select:

Inspect

Then select:

Console

- ☐ Copy the complete first red error into your error log.
- ☐ Think about the last file you changed before the problem appeared.
- ☐ Compare that file with the lesson requirements.
- ☐ If necessary, restore the most recent working backup.

When asking AI for help, provide:

-
- What you were trying to do
 - What you expected to happen
 - What actually happened
 - The complete error message
 - The names of the files involved

Ask AI to return complete corrected files.

Do not ask for snippets.

MY ERROR LOG

Use this page whenever something does not work.

Writing down the problem helps you investigate it carefully and remember the solution.

ERROR 1

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

ERROR 2

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

CHAPTER INTRODUCTION

Before a business owner creates an account, they need to understand what the software does and why it may be useful.

The public website is the first part of the application visitors will see.

It will explain the problems caused by poor customer record management and show how the software helps a business keep customer information organised.

The landing page will also provide clear Login and Register links that will later connect visitors to the authentication system.

In this chapter, you will build the complete public website using three files:

index.html

styles.css

script.js

The HTML file will contain the structure.

The CSS file will control the appearance.

The JavaScript file will control the responsive navigation.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- The project folder
- index.html
- styles.css

- script.js
- Responsive navigation
- Three-line hamburger menu
- Hero section
- Business problem section
- Features section
- How It Works section
- Customer dashboard preview
- Final call-to-action
- Footer
- Login link
- Register link
- Responsive mobile layout

By the end of the chapter, you will have a polished public website ready for the Supabase connection and authentication system.

LESSON 1

UNDERSTANDING THE CUSTOMER RECORD MANAGEMENT SYSTEM

Estimated Time

10 minutes

WHAT YOU ARE BUILDING

You are building software that helps a business manage customer information in one organised system.

A user will be able to create an account and store customer details such as:

- Name
- Phone number
- Email address
- Company
- Job title
- Address
- Location
- Notes

The software will also allow the user to search, filter, sort, update, and delete customer records.

Every user's customer information will remain private.

WHY THIS MATTERS

Many small businesses do not have a reliable customer record system.

Customer information may be stored inside phone contacts, notebooks, messaging applications, spreadsheets, receipts, or the memory of one employee.

This creates several problems.

The business may struggle to find a customer's number.

Two employees may create the same customer record.

Important notes may be forgotten.

Customer information may be lost when a phone is replaced.

The business may also find it difficult to understand how many customers it has or how quickly the customer list is growing.

A Customer Record Management System solves these problems by keeping customer information in one structured database.

BEFORE YOU CONTINUE

Create a new folder on your computer.

Name it exactly:

Customer Record Management System

This folder will contain every file used throughout Workbook 02.

Do not copy or save files from any other workbook inside this folder.

Each software project should have its own folder.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the project folder and make sure you understand what the completed application will do.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson prepares you for the project.

SAVE YOUR FILES

No files are created during this lesson.

Confirm that the folder is named:

Customer Record Management System

TEST YOUR WORK

Open the new folder.

It should be empty.

Do not copy files from any other workbook into this folder.

You will build this project from the beginning.

CHECKPOINT

Before moving on, confirm that:

- ☒ You created the Customer Record Management System folder.
- ☒ The folder is empty.
- ☒ You understand the main purpose of the software.
- ☒ You understand that every user will have private customer records.

COMMON BEGINNER MISTAKES

One common mistake is placing files from different projects inside the same folder.

This can cause the browser to load the wrong stylesheet or JavaScript file.

Keep this project completely separate from any other workbook.

Another mistake is renaming the folder repeatedly during the project.

Choose the correct name now and keep it unchanged.

BEHIND THE SCENES

The application will eventually contain several connected systems.

The public website will explain the software.

Authentication will identify the user.

The customers table will store records.

Row Level Security will protect those records.

JavaScript will connect the forms and dashboard to Supabase.

Every part will be added in the correct order.

THINK LIKE A SOFTWARE DESIGNER

Before building software, understand the real problem it is expected to solve.

This project is not simply a form that saves names and phone numbers.

It is a business system designed to make customer information easier to store, find, update, and use.

That difference should guide every feature you build.

WHAT YOU LEARNED

In this lesson you learned:

- what a Customer Record Management System does
- why businesses need organised customer records
- what information the software will store
- why every software project should have its own folder
- why the application must protect each user's data

In the next lesson, you will build the first version of the public landing page.

CHAPTER 1**BUILDING THE PUBLIC WEBSITE****LESSON 2**

BUILDING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the first version of the public website for your Customer Record Management System.

The landing page will explain what the software does, the business problems it solves, and the main features users will receive.

At this stage, the page will contain the complete HTML structure.

It will not look polished yet because the stylesheet has not been created.

The responsive navigation will also not work until the JavaScript file is added in a later lesson.

WHY THIS MATTERS

A business software application needs a clear public website.

Before someone creates an account, they should be able to understand:

- What the software does
- Who it is designed for
- What problems it solves
- What features are available
- What action they should take next

A well-structured landing page helps visitors understand the value of the application before they register.

BEFORE YOU CONTINUE

Confirm that your project folder is named:

Customer Record Management System

Open the folder.

It should still be empty.

You are about to create the first file:

index.html

When saving the file in Notepad:

- Click File.
- Click Save As.
- Open the Customer Record Management System folder.
- Change Save as type to All Files.
- Enter the filename exactly as:

index.html

Make sure the file is not saved as:

index.html.txt

BUILD PROMPT

PROMPT STARTS HERE

Create one complete HTML5 file named:

index.html

The file is the public landing page for a Customer Record Management System designed for small businesses.

Return the complete index.html file only.

Do not return explanations.

Do not return snippets.

Do not return CSS.

Do not return JavaScript.

Do not omit any section.

FILE CONNECTIONS

Inside the head section, include this stylesheet link exactly:

```
<link rel="stylesheet" href="styles.css">
```

Immediately before the closing body tag, include this JavaScript link exactly:

```
<script src="script.js"></script>
```

The files styles.css and script.js will be created in later lessons.

Do not include CSS inside index.html.

Do not include JavaScript inside index.html.

GENERAL HTML REQUIREMENTS

- Use proper HTML5 structure.
- Include the viewport meta tag.
- Include an appropriate page title.
- Use semantic HTML elements.
- Use clear and meaningful class names.
- Use section IDs that match the navigation links.
- Keep the content professional and easy to understand.
- Do not use external images.
- Do not use external icons.
- Do not use any framework.
- Do not use placeholder text such as Lorem Ipsum.

PAGE STRUCTURE

Build the sections below in this exact order.

1. HEADER AND NAVIGATION

Create a professional header.

Include a clickable logo that displays:

Customer Records

The logo must link to:

index.html

Include these navigation links:

Home

Features

How It Works

Login

Register

The links should point to:

Home:

#home

Features:

#features

How It Works:

#how-it-works

Login:

login.html

Register:

register.html

Style-related classes should already be added so CSS can later make Login a secondary action and Register the main action.

Add a mobile hamburger button.

The hamburger button must:

- Use a button element.
- Include exactly three empty span elements.
- Include an aria-label that explains its purpose.
- Include:

aria-expanded="false"

- Include:

aria-controls

pointing to the navigation menu.

Do not build the hamburger lines using CSS pseudo-elements.

The three span elements must exist inside the HTML.

2. HERO SECTION

Give the section the ID:

home

Include:

- A clear headline explaining that the software helps businesses organise customer information.
- A short supporting paragraph.
- A primary button labelled:

Get Started

The button must link to:

register.html

- A secondary button labelled:

See How It Works

The button must link to:

#how-it-works

Also include a customer management preview built entirely with HTML.

The preview should contain simple cards or panels representing:

- Total Customers
- New Customers This Month
- Business Customers
- A short customer list

Do not use an image.

3. BUSINESS PROBLEM SECTION

Create a section explaining the problems businesses face when customer information is scattered across:

- Notebooks
- Phone contacts
- Spreadsheets

- Messaging applications
- Employees' personal devices

Explain that scattered records can lead to:

- Lost customer details
- Duplicate records
- Missed follow-ups
- Slow customer service
- Difficulty understanding customer growth

Keep the wording clear and concise.

4. FEATURES SECTION

Give the section the ID:

features

Include six feature cards.

The cards should explain:

1. Secure Customer Records
2. Duplicate Detection
3. Customer Profile Cards
4. Click-to-Call
5. Click-to-Email
6. Search, Filters and Sorting

Each card should include:

- A short title
- A brief explanation

Do not use external icons.

You may use simple text labels or small decorative HTML elements.

5. HOW IT WORKS SECTION

Give the section the ID:

how-it-works

Explain the process in four clear steps:

1. Create an Account

2. Add Customer Details

3. Manage Customer Records

4. Find and Contact Customers Quickly

Each step should have:

- A number
- A heading
- A short explanation

6. DASHBOARD PREVIEW SECTION

Create a larger dashboard preview using HTML only.

Include:

- Four summary cards
- A customer search area
- Filter controls
- Three sample customer profile cards

Each sample customer card should display example information such as:

- Customer initials
- Customer name
- Customer type
- Company
- Phone
- Email
- View Profile button

The sample data must be clearly fictional.

Do not use real personal information.

The buttons are only part of the visual preview and do not need to work.

7. FINAL CALL-TO-ACTION SECTION

Encourage the visitor to organise customer records in one secure system.

Include a button labelled:

Create Your Account

The button must link to:

register.html

8. FOOTER

Include:

- Logo or software name
- A short description
- Quick links
- Login link pointing to login.html
- Register link pointing to register.html
- Copyright text

Use a JavaScript-friendly span with an ID for the copyright year if appropriate, but do not include JavaScript inside the page.

ACCESSIBILITY REQUIREMENTS

- Use heading levels in a logical order.
- Add descriptive text to links and buttons.
- Use aria-labels where necessary.
- Make the hamburger button accessible.
- Use meaningful link text.
- Avoid empty links.

IMPORTANT

The Login links must point to:

login.html

The Register and Get Started links must point to:

register.html

The stylesheet link must be:

```
<link rel="stylesheet" href="styles.css">
```

The JavaScript link must be:

```
<script src="script.js"></script>
```

Return one complete index.html file from beginning to end.

Do not include unfinished comments.

Do not return any other file.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

index.html

The file should contain:

- ☒ Correct HTML5 structure
- ☒ Stylesheet link to styles.css
- ☒ JavaScript link to script.js
- ☒ Header and navigation
- ☒ Three-span hamburger button
- ☒ Hero section
- ☒ Business problem section
- ☒ Six feature cards
- ☒ Four-step How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer
- ☒ Login links pointing to login.html
- ☒ Register links pointing to register.html

The response should begin with:

```
<!DOCTYPE html>
```

It should not begin with an explanation.

SAVE YOUR FILES

Copy the complete HTML code returned by ChatGPT.

Open Notepad.

Paste the complete code into the empty document.

Click File.

Click Save As.

Open your Customer Record Management System folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

index.html

Click Save.

Close Notepad.

Open the project folder and confirm that the file appears as:

index.html

Make sure it is not named:

index.html.txt

TEST YOUR WORK

Open your Customer Record Management System folder.

Double-click:

index.html

The page should open in your browser.

It will look plain because styles.css does not exist yet.

That is expected.

Scroll through the complete page.

Confirm that you can see:

- ☒ Header
- ☒ Navigation
- ☒ Hero section
- ☒ Business problem section
- ☒ Features section
- ☒ How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer

Click:

Features

The browser should move to the Features section.

Click:

How It Works

The browser should move to the How It Works section.

Click:

Login

The browser may show that login.html does not exist.

This is expected.

Click:

Register

The browser may show that register.html does not exist.

This is also expected.

Those pages will be created later as part of the complete authentication system.

CHECKPOINT

Before moving on, confirm that:

- ✓ index.html exists.
- ✓ The browser opens the file as a webpage.
- ✓ The page does not display HTML code as plain text.
- ✓ Every required section is present.
- ✓ The hamburger button contains three span elements.
- ✓ Login points to login.html.
- ✓ Register points to register.html.
- ✓ styles.css is linked.
- ✓ script.js is linked.
- ✓ The filename is not index.html.txt.

COMMON BEGINNER MISTAKES

A common mistake is saving the file using the wrong extension.

If the file is saved as:

index.html.txt

the browser may not treat it as a webpage.

Open the project folder and check the complete filename carefully.

Another mistake is removing the links to styles.css and script.js because those files do not exist yet.

Do not remove them.

They are being linked now so the files will connect correctly when they are created.

Some students also change:

login.html

to:

login

or change:

register.html

to:

signup.html

Do not do this.

The filenames must remain consistent because later pages and redirects will depend on them.

BEHIND THE SCENES

The landing page already refers to files that will be created in later lessons.

This is intentional.

The browser reads:

```
<link rel="stylesheet" href="styles.css">
```

and looks for a file named styles.css inside the same folder.

It also reads:

```
<script src="script.js"></script>
```

and looks for script.js inside the same folder.

The page remains usable even though those files are not present yet, but it will not have its final appearance or interactive navigation until they are created.

THINK LIKE A SOFTWARE DESIGNER

A public website should help visitors understand the software before asking them to create an account.

The sections should answer the visitor's questions in a sensible order.

First, explain the software.

Next, explain the business problem.

Then show the features and how the system works.

Finally, ask the visitor to register.

This creates a clear journey instead of presenting disconnected information.

CODE-READING QUESTION

Open index.html. Find the part of the page added for building the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create the main HTML file for a business application
- structure a professional landing page
- connect future CSS and JavaScript files correctly
- prepare navigation for future authentication pages
- build a hamburger button with three span elements
- save an HTML file correctly using Notepad

In the next lesson, you will create styles.css and transform the plain landing page into a polished, responsive business website.

CHAPTER 1**BUILDING THE PUBLIC WEBSITE****LESSON 3**

STYLING THE LANDING PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the stylesheet for your Customer Record Management System.

The HTML structure is already complete.

You will now transform it into a professional business website suitable for a real software product.

The responsive navigation will be completed in the next lesson when you create the JavaScript file.

WHY THIS MATTERS

Without CSS, a webpage contains only structure.

CSS controls:

- Colours
- Typography
- Layout
- Spacing
- Buttons
- Cards
- Responsive behaviour

A professional design helps visitors understand the software more quickly and creates confidence in the product.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ✓ index.html opens correctly.
- ✓ Every landing page section appears.
- ✓ The browser does not display HTML code as plain text.

Open index.html in Notepad.

Copy the complete code.

You will paste it into the Build Prompt so ChatGPT can style the exact HTML structure you have already created.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete HTML file named:

index.html

for my Customer Record Management System.

Create one complete CSS file named:

styles.css

Return only the complete styles.css file.

Do not return HTML.

Do not return JavaScript.

Do not return explanations.

Do not return snippets.

Everything already built must continue working.

The stylesheet must match the exact HTML structure that I will paste below.

The HTML already contains this stylesheet link:

```
<link rel="stylesheet" href="styles.css">
```

GENERAL DESIGN

Create a clean, modern business software design.

Use a professional colour palette suitable for business applications.

Create CSS variables inside:

:root

Include variables for:

- Primary colour
- Secondary colour
- Accent colour
- Background colour
- Card colour
- Text colour
- Border colour
- Shadow
- Border radius
- Content width

Use:

box-sizing: border-box;

for every element.

Remove unnecessary browser margins and padding.

Use a professional system font stack.

Do not import fonts.

GENERAL LAYOUT

Create a centred content container.

Give each major section comfortable spacing.

Prevent content from touching the edges of the screen.

Create balanced spacing between headings, paragraphs, cards and buttons.

HEADER

Create a professional header.

Display:

Customer Records

clearly as the brand.

Arrange navigation horizontally on larger screens.

Style:

Login

as a secondary button.

Style:

Register

as the primary action.

Include hover and keyboard focus styles.

HAMBURGER BUTTON

Style the existing hamburger button.

The button already contains exactly three span elements.

Do not create additional lines using CSS pseudo-elements.

Hide the hamburger button on larger screens.

Display it on smaller screens.

Prepare the navigation menu so it becomes visible whenever JavaScript adds an:

active

class.

Do not write JavaScript.

HERO SECTION

Create an attractive hero layout.

Display the content and dashboard preview side by side on wider screens.

Stack them on smaller screens.

Make:

Get Started

the strongest visual button.

Style:

See How It Works

as a secondary button.

BUSINESS PROBLEM SECTION

Give this section a subtle visual difference from the hero section.

Use generous spacing.

Make the problem easy to read.

FEATURES SECTION

Display the six feature cards in a responsive grid.

Each card should include:

- Rounded corners
- Professional shadow
- Comfortable spacing
- Subtle hover effect

HOW IT WORKS SECTION

Display the four steps using attractive cards.

Clearly distinguish the step number from the description.

DASHBOARD PREVIEW

Style the HTML dashboard preview so it resembles a real business dashboard.

Style:

- Summary cards
- Search area
- Customer profile cards
- View Profile buttons

Do not depend on images.

FINAL CALL-TO-ACTION

Create a strong closing section.

Highlight the:

Create Your Account

button.

FOOTER

Create a professional footer.

Arrange footer content neatly.

Style footer navigation links.

RESPONSIVE DESIGN

Create responsive layouts for:

Desktop

Tablet

Mobile

On smaller screens:

Hide the desktop navigation.

Display the hamburger button.

Prepare the navigation to become visible when JavaScript adds the:

active

class.

Stack multi-column layouts vertically.

Prevent horizontal scrolling.

Keep buttons large enough to tap comfortably.

ACCESSIBILITY

Create visible keyboard focus styles.

Maintain good colour contrast.

Respect reduced-motion preferences where appropriate.

TECHNICAL REQUIREMENTS

Match the exact class names used in the supplied HTML.

Do not invent a different HTML structure.

Do not use CSS frameworks.

Do not use external fonts.

Return one complete:

styles.css

file.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

styles.css

The file should:

- ☒ Match the HTML.
- ☒ Style every section.
- ☒ Support desktop, tablet and mobile layouts.
- ☒ Prepare the navigation for JavaScript.
- ☒ Include no HTML.
- ☒ Include no JavaScript.

SAVE YOUR FILES

Copy the complete CSS returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Customer Record Management System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

styles.css

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

TEST YOUR WORK

Open:

index.html

If it is already open in your browser, refresh the page.

The landing page should now have a professional appearance.

Confirm that:

- ☒ Header is styled.
- ☒ Navigation is styled.
- ☒ Hero section looks professional.
- ☒ Dashboard preview resembles business software.
- ☒ Feature cards display correctly.
- ☒ Buttons look professional.
- ☒ Footer is styled.

Now reduce the browser width.

Confirm that:

- ☒ Layout adjusts correctly.

☒ Hamburger button appears.

☒ Desktop navigation disappears.

The hamburger button does not need to work yet.

That functionality will be added in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ styles.css exists.

☒ The landing page is styled.

☒ Responsive layouts work.

☒ Hamburger button appears on smaller screens.

☒ Buttons display correctly.

☒ Dashboard preview looks professional.

☒ Footer is styled.

COMMON BEGINNER MISTAKES

A common mistake is saving the file as:

styles.css.txt

Always choose:

Save as type:

All Files

Another common mistake is changing class names inside the CSS.

The stylesheet must match the HTML structure exactly.

If ChatGPT returns styles that do not match your HTML, provide your complete index.html again and ask for a complete updated styles.css file.

Do not ask for snippets.

BEHIND THE SCENES

When your browser opens index.html, it immediately reads:

```
<link rel="stylesheet" href="styles.css">
```

It then loads every CSS rule inside styles.css and applies those rules to matching HTML elements.

This is why HTML and CSS must always stay in sync.

If an HTML class changes but the CSS is not updated, the styling may stop working.

THINK LIKE A SOFTWARE DESIGNER

Business software should communicate trust.

Professional colours, balanced spacing, consistent buttons, and organised layouts help users feel confident before they even create an account.

Good design is not decoration.

Good design makes software easier to understand and easier to use.

CODE-READING QUESTION

Open styles.css. Find one style rule added for styling the landing page. Which selector does the rule use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a professional stylesheet
- style a business landing page
- prepare responsive layouts

- style dashboard preview cards
- prepare the navigation for JavaScript
- keep HTML and CSS working together

In the next lesson, you will create `script.js` and make the responsive navigation fully functional.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 4

ADDING THE RESPONSIVE NAVIGATION

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the JavaScript that makes the responsive navigation work.

Your landing page already contains:

- HTML structure
- Professional styling
- A three-line hamburger button

The only missing piece is the behaviour.

When the browser window becomes narrow, visitors should be able to open and close the navigation menu using the hamburger button.

WHY THIS MATTERS

Many people will visit your website using a mobile phone or tablet.

A navigation menu that works well on a large computer screen may become difficult to use on a smaller device.

Responsive navigation solves this problem by allowing users to open and close the menu whenever they need it.

This creates a cleaner layout and improves the overall user experience.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that your project folder contains:

index.html

styles.css

Open index.html in your browser.

Reduce the browser width.

You should see:

☒ The hamburger button.

☒ The normal navigation hidden.

The hamburger button will not do anything yet.

That is expected.

Open index.html in Notepad.

Copy the complete code.

Also open styles.css in Notepad and copy the complete code.

You will paste both files into the Build Prompt so ChatGPT can use the exact IDs and class names already in your project.

BUILD PROMPT

PROMPT STARTS HERE

I already have these files in my project:

index.html

styles.css

Everything already built must continue working.

Create one complete JavaScript file named:

script.js

Return only the complete script.js file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

- Use plain Vanilla JavaScript.
- Do not use any framework.
- Do not use external libraries.
- Do not use Supabase.
- Do not add authentication code.
- Do not add customer management code.

RESPONSIVE NAVIGATION

- Use the exact HTML structure that I provide.
- Find:
 - The hamburger button.
 - The navigation menu.
- Use the exact IDs or class names already present.
- When the hamburger button is clicked:
 - Open the navigation menu.
 - Add the existing active class expected by styles.css.
- When the hamburger button is clicked again:
 - Close the navigation menu.
 - Remove the active class.

ACCESSIBILITY

- Update:
 - aria-expanded
 - correctly.
- When the menu opens:
 - aria-expanded="true"
- When the menu closes:
 - aria-expanded="false"
- Do not remove any accessibility attributes already present.

CLOSING THE MENU

When the visitor clicks any navigation link inside the mobile menu:

- Close the menu.

When the visitor presses the Escape key:

- Close the menu.

When the visitor clicks anywhere outside the navigation:

- Close the menu.

When the browser window becomes wider than the mobile layout:

- Close the mobile menu automatically.

SMOOTH NAVIGATION

Allow links pointing to sections inside:

`index.html`

to move smoothly.

Do not interfere with links pointing to:

`login.html`

`register.html`

Those links must continue opening normally.

CODE QUALITY

Wait until the page has loaded before selecting HTML elements.

Check that required elements exist before attaching event listeners.

Use clear variable names.

Avoid duplicate event listeners.

Do not include unfinished comments.

Return one complete `script.js` file.

IMPORTANT

Everything already built must continue working.

Use the exact IDs and class names already present inside my HTML.

Do not invent different selectors.

The active class must match the class already prepared inside styles.css.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

Here is my complete styles.css file:

[PASTE YOUR COMPLETE STYLES.CSS HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

script.js

The file should:

- ☒ Open the mobile navigation.
- ☒ Close the mobile navigation.
- ☒ Update aria-expanded.
- ☒ Close after selecting a navigation link.
- ☒ Close after pressing Escape.
- ☒ Close after clicking outside the navigation.
- ☒ Close automatically when returning to desktop width.
- ☒ Leave Login and Register links working normally.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Customer Record Management System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

script.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

Make sure the file is not named:

script.js.txt

TEST YOUR WORK

Open:

index.html

If it is already open, refresh the browser.

Reduce the browser width until the hamburger button appears.

Click the hamburger button.

The navigation menu should open.

Click the hamburger button again.

The navigation menu should close.

Open the menu.

Click:

Features

The menu should close automatically.

The browser should move to the Features section.

Open the menu again.

Press the Escape key.

The menu should close.

Open the menu again.

Click outside the navigation.

The menu should close.

Finally, widen the browser window.

The mobile menu should close automatically and the normal desktop navigation should appear again.

Click:

Login

The browser may display a page-not-found message.

This is expected because login.html has not yet been created.

Click:

Register

The browser may also display a page-not-found message.

This is expected because register.html will be created later in Workbook 02.

CHECKPOINT

Before moving on, confirm that:

- ☒ script.js exists.
- ☒ The hamburger button opens the menu.
- ☒ The hamburger button closes the menu.
- ☒ Navigation links close the menu.
- ☒ Escape closes the menu.

- ✓ Clicking outside closes the menu.
- ✓ Returning to desktop width closes the mobile menu.
- ✓ Login still points to login.html.
- ✓ Register still points to register.html.

COMMON BEGINNER MISTAKES

A common mistake is changing the class name used by JavaScript without updating the CSS.

The JavaScript and CSS must both use the same active class.

Another common mistake is searching for HTML elements using IDs or class names that do not exist.

Always use the exact HTML already present in your project.

If the menu does not open, provide ChatGPT with both your complete index.html and styles.css files and ask it to regenerate the complete script.js file.

Do not ask for only a small correction.

BEHIND THE SCENES

JavaScript does not create a second navigation menu.

Instead, it changes the state of the menu that already exists inside index.html.

When the active class is added, CSS displays the menu.

When the active class is removed, CSS hides it again.

The browser simply switches between those two states.

THINK LIKE A SOFTWARE DESIGNER

Responsive navigation should feel natural.

Users should never wonder how to close the menu or whether the application has stopped responding.

Good software provides predictable behaviour in every situation.

Small details like automatically closing the menu after a navigation link is selected make an application feel much more polished.

CODE-READING QUESTION

Open script.js. Find one function that supports adding the responsive navigation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a JavaScript file
- control responsive navigation
- update accessibility attributes
- respond to user interactions
- coordinate HTML, CSS and JavaScript
- improve the mobile experience

In the next lesson, you will review the complete public website, improve its overall quality, and prepare it for the Supabase connection in Chapter 2.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 5

COMPLETING THE PUBLIC WEBSITE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will review and improve the complete public website for your Customer Record Management System.

You have already built:

- `index.html`
- `styles.css`
- `script.js`

Your website is functional, but like most software, it can still be improved.

Rather than adding new features, you will refine what already exists while preserving every working part of the project.

WHY THIS MATTERS

Professional software is rarely perfect after the first version.

Developers continuously improve layouts, wording, accessibility, responsiveness, spacing, and usability.

These improvements make the software easier to understand and more pleasant to use without changing its purpose.

Learning how to improve existing software safely is one of the most valuable software development skills.

BEFORE YOU CONTINUE

Confirm that your Customer Record Management System folder contains:

index.html

styles.css

script.js

Open index.html in your browser.

Confirm that:

☒ The landing page opens.

☒ The page is fully styled.

☒ The responsive navigation works.

☒ The Login link points to:

login.html

☒ The Register link points to:

register.html

Do not continue until everything above works correctly.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete public website for my Customer Record Management System.

Everything already built must continue working.

Do not remove any existing functionality.

Return complete updated versions of every affected file.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

index.html

styles.css

script.js

only if improvements are required.

GENERAL GOAL

Review the complete public website and improve its overall quality while preserving every existing feature.

INDEX.HTML

Keep every existing section.

Do not remove:

- Header
- Navigation
- Hero
- Business Problem
- Features
- How It Works
- Dashboard Preview
- Final Call-to-Action
- Footer

Improve where appropriate:

- Heading wording
- Paragraph wording
- Accessibility labels
- Semantic HTML

Do not rename IDs unless absolutely necessary.

Keep:

```
<link rel="stylesheet" href="styles.css">
```

Keep:

```
<script src="script.js"></script>
```

Continue pointing:

Login

to:

login.html

Continue pointing:

Register

to:

register.html

STYLES.CSS

Keep every existing style.

Improve where appropriate:

- Consistent spacing
- Typography
- Button styling
- Card alignment
- Section spacing
- Mobile layout
- Dashboard preview appearance
- Footer layout

Improve keyboard focus styles where appropriate.

Maintain good colour contrast.

Do not redesign the page completely.

Refine the existing design.

SCRIPT.JS

Keep every existing feature working.

Review the JavaScript for:

- Duplicate event listeners
- Repeated logic
- Unnecessary code
- Accessibility improvements

Continue supporting:

- Responsive navigation
- Hamburger menu
- Navigation closing
- Escape key

- Outside click
 - Desktop resize handling
- Do not add authentication code.
- Do not add Supabase code.
- Do not add customer management code.

QUALITY REVIEW

Review the complete website and ensure:

- Navigation feels smooth.
- Buttons are consistent.
- Messages and wording are professional.
- Layout spacing is balanced.
- Mobile experience is clean.
- Keyboard navigation still works.

IMPORTANT

- Everything already built must continue working.
- Do not remove features.
- Do not rename files.
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

- ChatGPT should return complete updated versions of any files that require improvement.
- Every returned file should be complete from beginning to end.
- Nothing that already works should stop working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

If ChatGPT returns an updated version of any file:

Open that file in Notepad.

Delete the existing contents.

Paste the complete updated code.

Click File.

Click Save.

Repeat this process for every updated file.

Do not create duplicate files.

Always replace the complete contents of the existing file.

Your project folder should still contain only:

index.html

styles.css

script.js

TEST YOUR WORK

Refresh:

index.html

Review the complete landing page.

Confirm that:

- ☒ The header still appears correctly.
- ☒ Navigation still works.
- ☒ The hamburger menu still works.
- ☒ Every section remains present.
- ☒ Buttons remain consistent.
- ☒ The dashboard preview still looks professional.
- ☒ The footer still appears correctly.

Now reduce the browser width.

Confirm that:

- ☒ The layout adjusts correctly.
- ☒ The hamburger menu still works.
- ☒ No content runs outside the page.
- ☒ No unnecessary horizontal scrolling appears.

Finally, click:

Home

Features

How It Works

Login

Register

Confirm that:

- ☒ Internal navigation still works.
- ☒ Login still points to login.html.
- ☒ Register still points to register.html.

CHECKPOINT

Before moving on, confirm that:

- ☒ The landing page looks professional.

- ✓ Every section remains present.
- ✓ Navigation still works.
- ✓ Responsive behaviour still works.
- ✓ Login and Register links remain correct.
- ✓ No existing functionality has been lost.

COMMON BEGINNER MISTAKES

Some beginners ask ChatGPT to improve only a small section of a file.

That often causes important code to disappear.

Always request the complete updated file.

Another common mistake is accepting improvements without testing the complete website afterwards.

Even small changes can accidentally affect responsive layouts or navigation.

Refresh the browser and test the whole page after every update.

BEHIND THE SCENES

As software projects become larger, developers rarely rebuild them from scratch.

Instead, they improve existing files while preserving the features that already work.

Throughout this workbook, you will continue following this same approach.

Each Build Prompt will protect everything already built while adding or refining only what is necessary.

THINK LIKE A SOFTWARE DESIGNER

Professional software evolves through continuous improvement.

Instead of asking,

"What can I build next?"

experienced developers often ask,

"What can I make better?"

Learning to improve existing software without breaking it is one of the habits that separates experienced developers from beginners.

CODE-READING QUESTION

Open index.html. Find the part of the page added for completing the public website. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an existing project
- improve a public website safely
- preserve existing functionality
- refine layouts and wording
- test a website after improvements

CHAPTER SUMMARY

Congratulations.

You have completed the public website for your Customer Record Management System.

Your application now has a professional landing page that explains the software clearly and guides visitors towards creating an account.

You also built a responsive navigation system that works across different screen sizes.

Most importantly, you have followed the same professional workflow introduced in Workbook 01.

You built the application one reliable step at a time, testing each stage before moving forward.

CHAPTER MILESTONE

By the end of Chapter 1, you have successfully built:

- ✓ Professional landing page
- ✓ Responsive navigation
- ✓ Three-line hamburger menu
- ✓ Hero section
- ✓ Business problem section
- ✓ Features section
- ✓ How It Works section
- ✓ Dashboard preview
- ✓ Final call-to-action
- ✓ Footer
- ✓ Login link
- ✓ Register link
- ✓ Responsive layout
- ✓ HTML, CSS and JavaScript working together

Your project folder should now contain:

index.html

styles.css

script.js

TRANSITION TO CHAPTER 2

Your public website is now complete.

Visitors can learn about your software and navigate through the landing page.

The next step is to connect your application to Supabase.

In Chapter 2, you will create your Supabase project, build a shared connection file, test the connection, and prepare the application for the complete authentication system that follows.

Once that foundation is complete, you will begin building secure user accounts and protected customer management features.

CHAPTER 2

CONNECTING TO SUPABASE

CHAPTER INTRODUCTION

Your public website is now complete.

Visitors can learn about your Customer Record Management System, understand how it works, and choose to register for an account.

The next step is to give your application a secure online backend.

That backend is Supabase.

Supabase will be responsible for:

- User authentication
- Customer data storage
- Database security
- Protecting every user's customer records

During this chapter, you will connect your website to your own Supabase project.

Although visitors will not register until Chapter 3, everything required to support registration and customer management will already be in place.

Think of this chapter as laying the foundation before constructing the building.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Create a Supabase project
- Locate the Project URL
- Locate the Publishable Key

- Create a shared connection file
- Build a connection test page
- Verify that your application communicates successfully with Supabase

By the end of the chapter, your project will be fully prepared for authentication and customer management.

LESSON 1

CREATING YOUR SUPABASE PROJECT

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a new Supabase project specifically for your Customer Record Management System.

Each workbook should have its own Supabase project.

Keeping projects separate makes them easier to manage and prevents one application from accidentally affecting another.

WHY THIS MATTERS

Your Customer Record Management System needs a secure online backend.

Without one, users cannot:

- Create accounts
- Sign in
- Store customer records
- Retrieve customer records later

Supabase provides all of these services.

BEFORE YOU CONTINUE

Make sure you have:

- ☒ A working internet connection.
- ☒ A web browser.
- ☒ Your Customer Record Management System folder.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create your Supabase project.

Follow these steps.

1.

Open your web browser.

2.

Visit:

<https://supabase.com>

3.

Create a free account if you do not already have one.

4.

Sign in.

5.

Click:

New Project

6.

Choose your organisation.

7.

Enter a project name.

For example:

Customer Record Management System

8.

Create a strong database password.

Store it somewhere safe.

Do not share it with anyone.

Your application will not use this password directly.

9.

Choose the region closest to you.

10.

Click:

Create New Project

Wait while Supabase prepares the project.

This may take several minutes.

Do not continue until the project is ready.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your web browser.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

After the project has been created successfully:

Open the project dashboard.

Confirm that:

- ☒ The project opens.
- ☒ The project status shows that it is ready.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your Supabase account exists.
- ☒ Your new project has been created.
- ☒ The project dashboard opens successfully.
- ☒ The project is ready.

COMMON BEGINNER MISTAKES

Some students close the browser before Supabase finishes creating the project.

Always wait until the project is fully ready.

Another common mistake is forgetting the database password.

Store it somewhere safe before leaving this page.

Remember that your website will not use this password.

Later lessons will use the Publishable Key instead.

BEHIND THE SCENES

When Supabase creates your project, it automatically prepares:

- A PostgreSQL database
- User authentication
- Storage services
- Security settings
- APIs

These services work together to support the software you are building.

THINK LIKE A SOFTWARE DESIGNER

Professional developers normally create a separate backend for each application.

Keeping projects separate reduces confusion, improves security, and makes future maintenance much easier.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a Supabase project
- prepare an online backend

- understand why every software project should have its own database

In the next lesson, you will locate the Project URL and Publishable Key that allow your Customer Record Management System to communicate securely with Supabase.

CHAPTER 2
CONNECTING TO SUPABASE**LESSON 2**

UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will locate the two pieces of information your Customer Record Management System needs to communicate with Supabase.

These are:

- The Project URL
- The Publishable Key

In the next lesson, you will place these values inside a reusable connection file that every page in your application will use.

WHY THIS MATTERS

Your website needs to know where your Supabase project is located before it can communicate with it.

The Project URL identifies your Supabase project.

The Publishable Key allows your website to communicate with that project securely.

These two values will be used throughout this workbook whenever your application:

- Registers a user
- Signs in a user
- Loads customer records
- Saves customer records
- Updates customer records

- Deletes customer records

BEFORE YOU CONTINUE

Confirm that:

- ☒ Your Supabase project has finished creating.
- ☒ You are signed in to Supabase.
- ☒ Your project dashboard is open.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, locate your Project URL and Publishable Key.

Follow these steps carefully.

1.

Open your Supabase project.

2.

From the left-hand menu, click:

Project Settings

3.

Click:

Data API

Locate:

Project URL

It will look similar to:

<https://your-project-id.supabase.co>

Copy the complete value.

Do not change it.

Next, locate:

Publishable Key

Copy the complete key.

Store both values somewhere safe.

You will use them in the next lesson.

IMPORTANT

Your Project URL must end with:

.supabase.co

Do not add:

/rest/v1/

The correct format is:

<https://your-project-id.supabase.co>

IMPORTANT

Use only the:

Publishable Key

Never use:

- Service Role Key
- Secret Key
- Database password

Only the Publishable Key belongs inside your project files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your Supabase dashboard.

SAVE YOUR FILES

No project files are created during this lesson.

Keep your Project URL and Publishable Key ready for the next lesson.

TEST YOUR WORK

Confirm that you have copied:

- ☒ Your Project URL
- ☒ Your Publishable Key

Check the Project URL carefully.

It should end with:

.supabase.co

There should be nothing after that.

CHECKPOINT

Before moving on, confirm that:

- ☒ You have located your Project URL.
- ☒ You have located your Publishable Key.
- ☒ You copied the Publishable Key.
- ☒ You did not copy a secret key.
- ☒ Your Project URL does not contain:

/rest/v1/

COMMON BEGINNER MISTAKES

One common mistake is copying the wrong key.

Supabase displays several different keys.

Only the Publishable Key should be used in this workbook.

Never use the Service Role Key.

Never use a Secret Key.

Those keys are intended for secure server environments and should never appear inside files that visitors can download through their web browser.

Another common mistake is copying the REST API address instead of the Project URL.

Always use the base Project URL ending with:

.supabase.co

BEHIND THE SCENES

Whenever your application communicates with Supabase, it sends requests to your Project URL.

Those requests include your Publishable Key.

Supabase uses this information to identify your project before processing the request.

Although the Publishable Key is visible inside your website, it does not give visitors unrestricted access to your data.

Authentication and Row Level Security will provide that protection later in this workbook.

THINK LIKE A SOFTWARE DESIGNER

Professional software separates public information from private information.

The Project URL and Publishable Key are intended to be used by browser-based applications.

Sensitive credentials remain on secure servers and are never exposed to visitors.

Understanding this difference is one of the foundations of building secure software.

WHAT YOU LEARNED

In this lesson you learned how to:

- locate the Project URL
- locate the Publishable Key
- understand the purpose of both values
- avoid using secret keys
- prepare your project for the shared Supabase connection

In the next lesson, you will create a reusable Supabase connection file that every page in your Customer Record Management System will use.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 3

CREATING YOUR SUPABASE CONNECTION FILE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a reusable JavaScript file that connects your Customer Record Management System to your Supabase project.

Rather than writing the connection code on every page, you will create one shared file named:

`supabase-config.js`

Every page that needs to communicate with Supabase will use this file.

This keeps your project organised and makes future updates much easier.

WHY THIS MATTERS

As software grows, several pages often need to communicate with the same database.

Instead of copying the connection code into every JavaScript file, professional developers create one shared connection file.

Whenever another page needs Supabase, it simply loads this file.

This approach reduces mistakes, keeps the project easier to maintain, and ensures every page uses exactly the same connection.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Your Supabase project has been created.

✓ You have copied your Project URL.

✓ You have copied your Publishable Key.

Your Customer Record Management System folder should currently contain:

index.html

styles.css

script.js

You are about to add a fourth file.

BUILD PROMPT

PROMPT STARTS HERE

Create one complete JavaScript file named:

supabase-config.js

Return only the complete file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

Create a reusable Supabase connection file.

Use the official Supabase browser client.

Create the client using this format exactly:

```
const supabaseClient = window.supabase.createClient(  
  "https://your-project-id.supabase.co",  
  "YOUR_PUBLISHABLE_KEY"  
);
```

Replace the placeholder values with my actual values.

Project URL:

PASTE YOUR PROJECT URL HERE

Publishable Key:

PASTE YOUR PUBLISHABLE KEY HERE

REQUIREMENTS

- Create only one Supabase client.
- Store it in a constant named:
`supabaseClient`
- Do not use:
`/rest/v1/`
inside the Project URL.
- Do not use:
Service Role Key
- Do not use:
Secret Key
- Do not use:
Database password
- Include a short comment at the top of the file explaining that this file creates the shared Supabase connection used throughout the application.
- Return one complete JavaScript file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`supabase-config.js`

The file should:

- ☒ Create one shared Supabase client.
- ☒ Use my Project URL.

☒ Use my Publishable Key.

☒ Create the client using:

```
const supabaseClient = window.supabase.createClient(...)
```

☒ Contain JavaScript only.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Open your Customer Record Management System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

supabase-config.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

Make sure the file is not named:

supabase-config.js.txt

TEST YOUR WORK

Open your Customer Record Management System folder.

Confirm that:

supabase-config.js

appears alongside the other project files.

Open the file in Notepad.

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key appears.

☒ The Supabase client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

At this stage, the connection file is not yet being used by any webpage.

That is expected.

The connection will be tested in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ supabase-config.js has been created.

☒ The filename is correct.

☒ The Project URL is correct.

☒ The Publishable Key is correct.

☒ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

One common mistake is copying the REST API address instead of the Project URL.

Always use the base URL ending with:

`.supabase.co`

Another common mistake is copying the Service Role Key instead of the Publishable Key.

Only the Publishable Key belongs inside this file.

Never place secret keys inside browser-based applications.

Some beginners also create another Supabase client later inside `register.js` or `dashboard.js`.

Do not do that.

Throughout this workbook, every page will use the single shared client created inside:

`supabase-config.js`

BEHIND THE SCENES

The line:

```
const supabaseClient = window.supabase.createClient(...)
```

creates one reusable connection between your application and your Supabase project.

Later in this workbook, every page that needs Supabase will simply use:

`supabaseClient`

instead of creating another connection.

This keeps the application consistent and greatly reduces the chance of connection errors.

THINK LIKE A SOFTWARE DESIGNER

Whenever several parts of an application need the same functionality, it is usually better to create that functionality once and reuse it everywhere.

This principle keeps software simpler, easier to maintain, and less likely to contain conflicting code.

Creating a shared Supabase connection file is one example of that idea.

CODE-READING QUESTION

Open supabase-config.js. Which line creates the Supabase client, and which two saved values does that line use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a shared Supabase connection file
- create one reusable Supabase client
- use the correct Project URL
- use the correct Publishable Key
- avoid exposing secret keys
- prepare your application for future database communication

In the next lesson, you will build a connection test page that confirms your Customer Record Management System can communicate successfully with your Supabase project.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 4

BUILDING A CONNECTION TEST PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a simple webpage that confirms your Customer Record Management System can communicate successfully with your Supabase project.

This page exists only for testing.

It is not part of the finished application.

Instead, it allows you to verify that your shared Supabase connection is working correctly before you begin building authentication.

WHY THIS MATTERS

Professional developers test important parts of an application before building additional features.

Imagine spending hours creating registration and login pages only to discover that your website was never connected to Supabase correctly.

Testing the connection now helps you identify problems while the project is still simple.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

You should also have:

- ☒ Your Supabase project
- ☒ Your Project URL
- ☒ Your Publishable Key

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have these files:

index.html

styles.css

script.js

supabase-config.js

Everything already built must continue working.

Do not modify any existing files.

Create one new HTML file named:

test-connection.html

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

GENERAL REQUIREMENTS

Create a complete HTML5 document.

Include:

- Appropriate page title
- Viewport meta tag

Inside the head section, include simple CSS that creates a clean, centred layout suitable for a testing page.

BODY CONTENT

Display:

Heading

Supabase Connection Test

Below the heading, display a short paragraph explaining that the page is checking communication with the Supabase project.

Create a status box.

The status box should initially display:

Checking connection...

Give this element the ID:

connection-status

SCRIPT LOADING ORDER

Immediately before the closing body tag, load these scripts in this exact order.

First:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

Second:

```
<script src="supabase-config.js"></script>
```

Third:

Create an inline JavaScript block that:

- Checks whether:

supabaseClient

exists.

If it exists:

Change the status box to display:

✅ Connected successfully to Supabase.

Apply a success style.

If it does not exist:

Display:

❌ Unable to connect to Supabase.

Apply an error style.

IMPORTANT

Use the existing:

supabase-config.js

Do not create another Supabase client.

Do not hard-code the Project URL.

Do not hard-code the Publishable Key.

Do not use authentication.

Do not create database tables.

Do not create users.

Return one complete HTML file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

test-connection.html

The page should:

- ☒ Load the Supabase browser library.
- ☒ Load supabase-config.js.
- ☒ Load scripts in the correct order.
- ☒ Check whether supabaseClient exists.
- ☒ Display a clear success or error message.

SAVE YOUR FILES

Copy the complete HTML returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Customer Record Management System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

test-connection.html

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TEST YOUR WORK

Open:

test-connection.html

The page should open in your browser.

After a moment, the message should change from:

Checking connection...

to:

✅ Connected successfully to Supabase.

If instead you see:

❌ Unable to connect to Supabase.

Do not continue.

Return to the previous lessons and carefully check:

- ✓ Project URL
- ✓ Publishable Key
- ✓ supabase-config.js
- ✓ Script loading order

CHECKPOINT

Before moving on, confirm that:

- ✓ test-connection.html opens.
- ✓ The page loads correctly.
- ✓ The Supabase browser library loads first.
- ✓ supabase-config.js loads second.
- ✓ The page displays:

Connected successfully to Supabase.

COMMON BEGINNER MISTAKES

The most common mistake is loading the JavaScript files in the wrong order.

The correct order is always:

1. Supabase browser library
2. supabase-config.js
3. Page-specific JavaScript

Another common mistake is creating another Supabase client inside the page.

Do not do this.

Your application should use only the shared client created inside:

supabase-config.js

BEHIND THE SCENES

This page does not create a new connection.

Instead, it loads the shared connection file.

If that file creates:

`supabaseClient`

successfully, the page simply confirms that the shared connection is available.

Every protected page you build later will use this same approach.

THINK LIKE A SOFTWARE DESIGNER

Professional software is built on reliable foundations.

Testing one important component before moving to the next makes larger projects much easier to debug.

Finding problems early almost always saves time later.

CODE-READING QUESTION

Open `test-connection.html`. Find the part of the page added for building a connection test page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a dedicated connection test page
- load the Supabase browser library
- reuse a shared connection file

- verify that your application can communicate with Supabase
- identify common connection problems

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 5

FINAL CONNECTION REVIEW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

This lesson is your final review before building the authentication system.

You are not creating any new files.

Instead, you will carefully inspect your project and confirm that every connection to Supabase has been configured correctly.

WHY THIS MATTERS

The authentication system depends completely on your Supabase connection.

If the connection is incorrect, registration, login, customer management and every future database feature will fail.

Confirming your work now helps prevent much larger problems later.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, review your project carefully.

Complete the following checks.

Open:

supabase-config.js

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key is being used.

☒ The shared client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

Next, open:

test-connection.html

Confirm that the scripts load in this exact order:

1.

Supabase browser library

2.

supabase-config.js

3.

The page's own JavaScript

Finally, open:

test-connection.html

Confirm that the page displays:

☒ Connected successfully to Supabase.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by reviewing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Landing Page

☒ index.html opens correctly.

☒ Navigation works.

☒ Responsive menu works.

Supabase

☒ Project created.

☒ Project URL correct.

☒ Publishable Key correct.

☒ Shared connection file created.

Connection Test

☒ test-connection.html opens.

☒ Connected successfully to Supabase appears.

Project Folder

☒ index.html

☒ styles.css

☒ script.js

✓ supabase-config.js

✓ test-connection.html

CHECKPOINT

Before moving to Chapter 3, confirm that:

✓ Every required file exists.

✓ The landing page still works.

✓ The connection test succeeds.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ The script loading order is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

Some students continue building after the connection test has failed.

Do not do this.

Authentication depends on the connection working correctly.

Solve connection problems now before moving to the next chapter.

BEHIND THE SCENES

At this point, your Customer Record Management System knows how to communicate with Supabase.

What it cannot do yet is identify users.

That is the purpose of the next chapter.

Authentication allows the application to recognise who is currently signed in so every customer record can be linked to the correct person.

THINK LIKE A SOFTWARE DESIGNER

Every large software project is built in layers.

Public website.

Backend connection.

Authentication.

Database.

Business features.

Building each layer carefully creates a much more reliable application than trying to build everything at once.

WHAT YOU LEARNED

In this lesson you learned how to:

- verify the shared Supabase connection
- review script loading order
- confirm that your application is ready for authentication
- understand why authentication comes before customer management

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System is now successfully connected to Supabase.

You created your own Supabase project, located the Project URL and Publishable Key, built a shared connection file, and confirmed that your application can communicate successfully with your online backend.

More importantly, you established a reusable architecture that every future page in the application will use.

Whenever a page needs Supabase, it will always load:

1. The Supabase browser library.
2. `supabase-config.js`.
3. The page-specific JavaScript file.

Maintaining this structure throughout the project will keep your application organised and reduce connection-related problems.

CHAPTER MILESTONE

By the end of Chapter 2, you have successfully completed:

- ✓ Supabase project
- ✓ Project URL
- ✓ Publishable Key
- ✓ Shared connection file
- ✓ Connection test page
- ✓ Connection verification
- ✓ Correct script loading order
- ✓ Backend preparation for authentication

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TRANSITION TO CHAPTER 3

Your public website is complete.

Your application is connected to Supabase.

The next step is to build the complete authentication system.

In Chapter 3, you will build registration, login, the protected dashboard, and the protected customer profile page as one coordinated system.

By creating every destination page together, every redirect will already have a valid destination, allowing the complete authentication workflow to function correctly from the very beginning.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

CHAPTER INTRODUCTION

Your Customer Record Management System now has a professional public website and a working connection to Supabase.

The next step is to allow people to create their own accounts and access their own customer records securely.

Authentication is one of the most important parts of any business application.

Without authentication, every visitor would have access to the same customer database.

That would make the software unsafe to use.

In this chapter, you will build the complete authentication system as one coordinated feature.

Instead of creating one page at a time, you will create every authentication page together.

This approach ensures that:

- Every redirect already has a destination.
- Every page is correctly linked.
- Every JavaScript file loads correctly.
- Every stylesheet is linked correctly.
- The complete user journey works from the very beginning.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- register.html
- login.html

- dashboard.html
- customer-profile.html
- auth.css
- register.js
- login.js
- dashboard.js
- customer-profile.js

You will also update:

- index.html

The completed authentication flow will work like this:

Visitor

↓

index.html

↓

Register

↓

register.html

↓

Email verification

↓

login.html

↓

Login

↓

dashboard.html

↓

Customer Profile

↓

customer-profile.html

↓

Logout

↓

login.html

Unauthenticated visitors who attempt to open:

dashboard.html

or

customer-profile.html

will be redirected automatically to:

login.html

LESSON 1

UNDERSTANDING THE AUTHENTICATION FLOW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

Before creating the authentication system, it is important to understand how every page works together.

Authentication is not simply a login page.

It is a complete journey that begins when a visitor creates an account and continues until they safely sign out.

Every page depends on another page.

That is why the entire system will be built together.

WHY THIS MATTERS

Many beginners build authentication one page at a time.

The result is often broken redirects, missing files and incomplete user journeys.

Professional developers think about the complete workflow before writing any code.

Doing so reduces mistakes and produces a much more reliable application.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

- ✓ The landing page works.
- ✓ The Supabase connection test succeeds.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, study the authentication flow below.

A visitor arrives at:

`index.html`

The visitor selects:

Register

The visitor opens:

`register.html`

The visitor creates an account.

Supabase sends a verification email.

The visitor verifies the email address.

The visitor returns to:

`login.html`

The visitor signs in.

The application redirects to:

`dashboard.html`

The user manages customer records.

Whenever the user opens a customer profile, the application opens:

`customer-profile.html`

When the user signs out, the application redirects to:

`login.html`

If the visitor is not authenticated and tries to open:

`dashboard.html`

or

`customer-profile.html`

the application immediately redirects them to:

```
login.html
```

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the authentication flow.

Make sure you understand where every page sends the user.

Every destination page already has a purpose.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Registration comes before Login.
- ☒ Login comes before Dashboard.
- ☒ Customer Profile is protected.
- ☒ Logout returns the user to login.html.
- ☒ Protected pages require authentication.

COMMON BEGINNER MISTAKES

One common mistake is thinking that a login page alone creates a complete authentication system.

A secure authentication system includes:

- Registration
- Email verification

- Login
- Protected pages
- Logout
- Automatic redirects

All of these parts work together.

BEHIND THE SCENES

Authentication allows the application to answer one important question:

Who is currently using the software?

Once that question has been answered, every customer record can be connected to the correct user.

THINK LIKE A SOFTWARE DESIGNER

Always think about the complete journey.

Instead of asking:

"What happens on this page?"

ask:

"Where did the user come from?"

and

"Where should the user go next?"

That way, every page becomes part of one connected application.

WHAT YOU LEARNED

In this lesson you learned:

- the complete authentication workflow
- how every authentication page connects together
- why protected pages exist
- why redirects are planned before building begins

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

Estimated Time

75 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete authentication system for your Customer Record Management System.

Rather than generating one page at a time, ChatGPT will generate every authentication page together.

This ensures that:

- Every destination page already exists.
- Every redirect already has somewhere to go.
- Every stylesheet is linked correctly.
- Every JavaScript file is loaded correctly.
- Everything already built continues working.

WHY THIS MATTERS

Registration, login, dashboard access and customer profiles are tightly connected.

Building them together reduces missing files, broken redirects and inconsistent behaviour.

This is the same professional approach you used successfully in Workbook 01.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

☒ The connection test displays:

Connected successfully to Supabase.

Do not continue until everything above is working.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything already built must continue working.

My project currently contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

The public website is complete.

The responsive navigation works.

The Supabase connection has been tested successfully.

Do not remove or break any existing functionality.

Return complete files only.

Do not return snippets.

Do not return explanations.

Return the following complete files:

1.

Updated index.html

2.

register.html

3.

login.html

4.

dashboard.html

5.

customer-profile.html

6.

auth.css

7.

register.js

8.

login.js

9.

dashboard.js

10.

customer-profile.js

GENERAL REQUIREMENTS

Everything already built must continue working.

Every HTML page must be complete.

Every HTML page must include proper HTML5 structure.

Every HTML page must include an appropriate page title.

Use semantic HTML.

Use modern, responsive layouts.

The logo on every page should display:

Customer Records

The logo should link to:

index.html

INDEX.HTML

Return a complete updated version.

Preserve:

- Hero
- Features
- Navigation
- Footer
- Responsive menu

Ensure Login points to:

login.html

Ensure Register points to:

register.html

REGISTER.HTML

Create a professional registration page.

Include:

- Logo
- Full Name
- Email Address
- Password
- Confirm Password
- Register button
- Link to login.html

Display friendly validation messages.

Disable the Register button while processing.

Display loading messages.

After successful registration:

Display a success message explaining that the user must verify their email before signing in.

Redirect the user to:

login.html

Do not automatically sign the user in.

LOGIN.HTML

Create a professional login page.

Include:

- Logo
- Email
- Password
- Login button
- Link to register.html

Display friendly validation and loading messages.

Disable the Login button while processing.

If login succeeds:

Redirect to:

dashboard.html

If the user is already authenticated:

Automatically redirect to:

dashboard.html

DASHBOARD.HTML

Create a professional dashboard page.

Include:

- Logo
- Welcome heading
- Logged-in user's email
- Four customer summary cards with temporary values:

Total Customers

Customers Added Today

New Customers This Month

Business Customers

Include a placeholder section where the customer management dashboard will be built later.

Include:

Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

CUSTOMER-PROFILE.HTML

Create a protected customer profile page.

Include:

- Logo
- Customer profile heading
- Placeholder customer information
- Back to Dashboard button
- Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

AUTH.CSS

Create one shared stylesheet for:

register.html

login.html

dashboard.html

customer-profile.html

Use:

Professional forms

Responsive layout

Business dashboard styling

Summary cards

Loading messages

Success messages

Error messages

REGISTER.JS

Load after:

1. Supabase CDN
2. supabase-config.js

Use:

```
const supabaseClient
```

already created inside:

```
supabase-config.js
```

Do not create another Supabase client.

Validate:

- Full Name
- Email
- Password
- Confirm Password

Passwords must match.

Create the account using Supabase Authentication.

When calling:

```
supabaseClient.auth.signUp
```

include an email confirmation destination.

Create it from the website that is currently open:

```
`${window.location.origin}/login.html`
```

Pass that address as:

```
emailRedirectTo
```

inside the sign-up options.

Do not hardcode a Netlify website name.

Using:

```
window.location.origin
```

allows the same complete files to work when the website address changes.

Display friendly success and error messages.

Disable the button while processing.

After successful registration:

Inform the user to open the verification email.

Redirect to:

login.html

Do not automatically sign the user in from the registration form.

LOGIN.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Authenticate users.

Display loading messages.

Display friendly errors.

Redirect successful logins to:

dashboard.html

DASHBOARD.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the user is authenticated.

Display the authenticated user's email.

Support Logout.

Redirect unauthenticated users to:

login.html

CUSTOMER-PROFILE.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the visitor is authenticated.

If not:

Redirect immediately to:

login.html

Support Logout.

Customer records will be loaded later in this workbook.

For now, create the protected page and its authentication logic.

SCRIPT LOADING ORDER

Every Supabase-powered HTML page must load:

1.

Supabase browser library

2.

supabase-config.js

3.

Its own page-specific JavaScript

IMPORTANT

Return every requested file completely.

Do not omit any file.

Do not return partial updates.

Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return ten complete files.

Every HTML page should already contain the correct stylesheet and JavaScript links.

Every protected page should already verify authentication.

Every redirect should already point to an existing page.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2 (CONTINUED)

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

ChatGPT should return ten complete files.

Work through them one at a time.

If the file already exists, replace its complete contents.

If the file does not already exist, create it.

Do not copy only part of a file.

Always replace the complete contents.

Updated index.html

Open:

index.html

in Notepad.

Select all the existing code.

Delete it.

Paste the complete updated code returned by ChatGPT.

Click:

File

Save

Close Notepad.

register.html

Open Notepad.

Create a new document.

Paste the complete code.

Click:

File

Save As

Open your Customer Record Management System folder.

Choose:

Save as type:

All Files

Enter:

register.html

Click Save.

login.html

Repeat the same process.

Save as:

login.html

dashboard.html

Repeat the same process.

Save as:

dashboard.html

customer-profile.html

Repeat the same process.

Save as:

customer-profile.html

auth.css

Repeat the same process.

Save as:

auth.css

register.js

Repeat the same process.

Save as:

register.js

login.js

Repeat the same process.

Save as:

login.js

dashboard.js

Repeat the same process.

Save as:

dashboard.js

customer-profile.js

Repeat the same process.

Save as:

customer-profile.js

When you finish, your project folder should contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

customer-profile.html

auth.css

register.js

login.js

dashboard.js

customer-profile.js

Check every filename carefully.

Do not allow any file to become:

register.html.txt

login.js.txt

dashboard.html.txt

or any similar variation.

TEST YOUR WORK

Do not assume the authentication system works.

Authentication must be tested from a website with an HTTPS address.

Do not test this chapter by double-clicking the HTML files in your project folder.

The files still remain inside your project folder.

You will simply place a test copy of the complete folder online before testing.

BEFORE YOU START THE TESTS

1.

Save every updated file in Notepad.

2.

Deploy the complete project folder to Netlify.

If you already created a Netlify test website for this project, deploy the updated folder to the same website.

3.

Copy the website address supplied by Netlify.

It should begin with:

`https://`

For example:

`https://your-site-name.netlify.app`

Your own website address will be different.

4.

Open your Supabase project.

Go to:

Authentication

Then open:

URL Configuration

5.

Set the Site URL to your Netlify website address.

Do not add a file name to the Site URL.

6.

Add this complete address to Redirect URLs:

`https://your-site-name.netlify.app/login.html`

Replace the example website name with your real Netlify website name.

7.

Make sure the latest version of every file has been deployed.

8.

Open the project from its Netlify HTTPS address.

Keep using that address throughout every test below.

TEST 1

PUBLIC WEBSITE

Open the deployed index.html page.

Confirm that:

- ☒ The landing page opens correctly.
- ☒ The page is fully styled.
- ☒ The responsive navigation still works.
- ☒ Login opens the deployed login.html page.
- ☒ Register opens the deployed register.html page.

Look at the browser address bar.

The address must begin with:

https://

Nothing already built should have stopped working.

TEST 2

REGISTRATION PAGE

Open the deployed register.html page.

Attempt to submit the form without entering any information.

Friendly validation messages should appear.

Complete the form using an email address that has not already been used for this test.

Click:

Register

The Register button should become disabled while the request is being processed.

A loading message should appear.

After successful registration:

You should see a message asking you to verify your email.

TEST 3

EMAIL VERIFICATION

Open your email inbox.

Locate the verification email from Supabase.

Open the email.

Click the verification link.

Supabase should verify the email and return the browser to your deployed website.

The returned address should use your Netlify website name.

It should not begin with:

file://

Depending on the authentication response, you may briefly see login.html or the application may recognise the new session and open the dashboard.

Both results confirm that the browser returned to the correct website.

If the login page remains open, sign in using the email address and password you registered.

If the verification email does not arrive immediately:

Wait a few minutes.

Refresh your inbox.

Check your Spam or Junk folder.

If the verification link opens the wrong website:

Return to Supabase URL Configuration.

Check the Site URL and Redirect URLs carefully.

Correct them before creating another test account.

TEST 4

LOGIN

Open the deployed login.html page.

Enter an incorrect password.

A friendly error message should appear.

Now enter the correct password.

Click:

Login

The Login button should become disabled while signing in.

A loading message should appear.

After a successful login:

The browser should open the deployed dashboard.html page.

TEST 5

AUTHENTICATED SESSION

After signing in successfully:

Confirm that the protected dashboard opens.

Confirm that your email address appears.

Refresh the page.

The dashboard should remain open because the same website still has your authenticated session.

Confirm that the four customer summary cards appear.

Confirm that the Logout button appears.

While signed in, open the complete deployed customer-profile.html address.

The protected customer profile page should open.

Click:

Logout

The browser should return to the deployed login.html page.

While signed out, open the complete deployed dashboard.html address.

The application should immediately return you to the deployed login.html page.

While still signed out, open the complete deployed customer-profile.html address.

The application should immediately return you to the deployed login.html page.

Sign in again.

While signed in, open the complete deployed login.html address.

The application should immediately return you to the deployed dashboard.html page.

CHECKPOINT

Before moving to the next lesson, confirm that:

- ☒ Registration works.
- ☒ Validation works.
- ☒ Loading messages appear.
- ☒ Email verification succeeds.
- ☒ Login works.
- ☒ Friendly error messages appear.
- ☒ Dashboard is protected.
- ☒ Customer Profile page is protected.
- ☒ Logout works.
- ☒ Authenticated users cannot remain on login.html.
- ☒ Unauthenticated users cannot open dashboard.html.
- ☒ Unauthenticated users cannot open customer-profile.html.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to verify the email address before attempting to sign in.

If email verification is enabled, Supabase will not allow the account to be used until verification has been completed.

Another common mistake is loading the JavaScript files in the wrong order.

Every Supabase-powered page should load:

1.

Supabase browser library

2.

supabase-config.js

3.

The page-specific JavaScript

Changing this order can prevent the application from working correctly.

Some beginners also create another Supabase client inside:

register.js

login.js

dashboard.js

or

customer-profile.js

Do not do this.

Every page should use the shared client created inside:

supabase-config.js

BEHIND THE SCENES

Your authentication system is now working as one connected application.

When a visitor creates an account, Supabase stores the authentication information securely.

After email verification, Supabase allows that account to sign in.

When login succeeds, Supabase creates a session.

Protected pages check whether that session exists before displaying their content.

If the session does not exist, the visitor is redirected immediately to the login page.

The Customer Profile page already uses this protection even though customer records have not yet been introduced.

Later, this same authentication system will protect every customer record in the application.

THINK LIKE A SOFTWARE DESIGNER

Authentication is more than allowing someone to sign in.

It controls who can enter the application, which pages they can open, and what information they are allowed to see.

Every protected page should perform its own authentication check.

Never assume that because a user reached one protected page, every other page is automatically secure.

CODE-READING QUESTION

Open `register.js`. Find one function that supports building the complete authentication system. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete authentication system
- connect registration, login and protected pages
- protect multiple destinations
- redirect unauthenticated visitors
- redirect authenticated users appropriately
- preserve every feature already built

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 3

AUDITING THE AUTHENTICATION SYSTEM

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not build new features.

Instead, you will carefully review every part of the authentication system before beginning customer management.

Authentication is the foundation of the entire application.

Every customer record created later will depend on the current user being identified correctly.

WHY THIS MATTERS

The next chapter introduces customer records.

Every customer record must belong to one authenticated user.

If authentication is unreliable, customer privacy cannot be guaranteed.

Professional developers verify security before adding business features.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

customer-profile.html

auth.css

register.js

login.js

dashboard.js

customer-profile.js

Confirm that you have successfully created at least one verified user account.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every authentication test again.

Do not continue until every test succeeds.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this final authentication check on the deployed HTTPS website.

Do not complete this audit from files opened with:

file://

SUPABASE URL SETTINGS

- ✓ The Site URL contains the Netlify website address.
- ✓ Redirect URLs contains the complete deployed login.html address.

REGISTRATION AND VERIFICATION

- ✓ Registration works from the deployed register.html page.
- ✓ A verification email arrives.
- ✓ The verification link returns to the deployed website.
- ✓ The returned address begins with https:// and uses the correct Netlify website name.

LOGIN AND SESSION

- ✓ Login works from the deployed login.html page.
- ✓ Refreshing a protected page keeps a signed-in user signed in.
- ✓ Logout works.

PROTECTION

- ✓ A signed-out visitor cannot remain on dashboard.html.
- ✓ A signed-in user can reach dashboard.html.
- ✓ A signed-in user who opens login.html returns to dashboard.html.

PROTECTED CUSTOMER PAGE

- ✓ A signed-in user can open customer-profile.html.
- ✓ A signed-out visitor cannot remain on customer-profile.html.

PAGES

- ✓ The signed-in user's email appears.
- ✓ The summary cards appear.

- ✓ The customer profile placeholder appears.

If every check passes, the authentication system is ready for the next chapter.

CHECKPOINT

Before moving to Chapter 4, confirm that:

- ✓ Every authentication page works.
- ✓ Every redirect works.
- ✓ Dashboard protection works.
- ✓ Customer Profile protection works.
- ✓ No authentication errors remain.

COMMON BEGINNER MISTAKES

Some students test only the successful login process.

Always test unsuccessful situations as well.

Protected pages should remain protected when no user is signed in.

Another common mistake is assuming one protected page automatically secures every other page.

Each protected page should verify authentication independently.

BEHIND THE SCENES

The authentication system now provides the identity that every future database operation will rely on.

When customer records are introduced in Chapter 4, every new customer will automatically belong to the currently authenticated user.

Authentication and Row Level Security will work together to protect customer information.

THINK LIKE A SOFTWARE DESIGNER

Security should be confirmed before adding business functionality.

A reliable authentication system makes every future feature easier to build because the application always knows who the current user is.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit an authentication system
- verify protected pages
- verify redirects
- prepare the application for secure customer management

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now includes a complete authentication system.

Visitors can register, verify their email addresses, sign in, access protected pages, and sign out securely.

Every protected page already knows how to verify the current user's session before displaying information.

Most importantly, your application can now identify who is currently using the system.

That capability will allow every customer record to belong to the correct user.

CHAPTER MILESTONE

By the end of Chapter 3, you have successfully built:

- ✓ User registration
- ✓ Email verification
- ✓ Secure login
- ✓ Protected dashboard
- ✓ Protected customer profile page
- ✓ Logout
- ✓ Friendly validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Shared authentication styling
- ✓ Complete authentication workflow

TRANSITION TO CHAPTER 4

Your Customer Record Management System can now identify every authenticated user.

The next step is to create the secure customer database.

In Chapter 4, you will design the customers table, create a richer business data model, enable Row Level Security, and prepare the application to store customer information safely.

Unlike Workbook 01, this workbook introduces a much richer database structure, allowing businesses to manage complete customer profiles instead of simple financial transactions.

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

CHAPTER INTRODUCTION

Your Customer Record Management System can now recognise who is signed in.

The next step is to create a secure database where each authenticated user can store their own customer records.

Unlike the Expense Tracker in Workbook 01, this application stores much richer information.

Instead of recording a single financial transaction, each customer record contains several related pieces of information.

For example, one customer record may include:

- Customer type
- First name
- Last name
- Email address
- Phone number
- Company
- Job title
- Address
- City
- State
- Country

- Notes

Managing this information properly requires better database design.

Throughout this chapter, you will create a database structure that is flexible enough for real businesses while remaining easy to understand.

You will also continue using Row Level Security so every authenticated user can access only their own customer records.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- customers table
- Rich customer data model
- Required and optional fields
- Primary Key
- User ownership
- Row Level Security
- SELECT policy
- INSERT policy
- UPDATE policy
- DELETE policy
- Database security testing

By the end of this chapter, your application will have a secure database capable of storing complete customer profiles.

LESSON 1

UNDERSTANDING THE CUSTOMER DATA MODEL

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

Before creating the database table, it is important to understand what information a business normally stores about its customers.

Not every customer record contains only a name and phone number.

Businesses often need additional information to provide better customer service.

This lesson explains the purpose of every field you will create later in this chapter.

WHY THIS MATTERS

Good software begins with good data design.

If the database is designed poorly, the software becomes difficult to improve later.

Professional developers spend time deciding what information should be stored before creating the database itself.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works.
- ☒ Login works.
- ☒ Dashboard protection works.
- ☒ Your Supabase project is open.

No changes will be made to your project files during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, study the customer data model below.

Every customer record will contain the following information.

Customer Type

Individual

or

Business

First Name

Required

Last Name

Required

Email Address

Optional

Phone Number

Required

Company

Optional

Job Title

Optional

Address

Optional

City

Optional

State

Optional

Country

Optional

Notes

Optional

Created At

Automatically created

Updated At

Automatically updated

Notice that some fields are required while others are optional.

The application should not force users to enter information they do not have.

However, it should always collect enough information to identify and contact a customer.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the complete customer data model.

Ask yourself:

Which fields are required?

Which fields are optional?

Why might a business customer include a company name while an individual customer does not?

If you can answer these questions, you are ready for the next lesson.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Required fields
- ☒ Optional fields
- ☒ Why customer records contain richer information than expense records

COMMON BEGINNER MISTAKES

Some beginners make every field required.

This often frustrates users because they may not know every piece of information when creating a new customer.

Another common mistake is collecting too little information.

A customer database should contain enough detail to make future communication easier.

BEHIND THE SCENES

This database has been designed to balance simplicity with practical business use.

It collects enough information to support customer management without becoming unnecessarily complicated.

Later workbooks will build on these same design principles.

THINK LIKE A SOFTWARE DESIGNER

Every database field should have a purpose.

Before adding a new field, ask yourself:

"Will this information help the business make better decisions or provide better service?"

If the answer is no, the field may not belong in the database.

WHAT YOU LEARNED

In this lesson you learned:

- how customer information is organised
- the difference between required and optional fields
- why database design matters
- how business software stores richer information

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 2

CREATING THE CUSTOMERS TABLE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the database table that stores every customer record.

Each row inside the table will represent one customer.

Every customer will belong to one authenticated user.

Later, the dashboard will use this table to create, display, update and delete customer records.

WHY THIS MATTERS

Without a properly designed table, your application has nowhere to store customer information.

Building the correct structure now makes every later feature much easier to develop.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Authentication is working.
- ☒ Your Supabase project is open.
- ☒ You understand the customer data model.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the customers table directly inside Supabase.

Open:

Table Editor

Click:

Create a new table

TABLE NAME

customers

DESCRIPTION

(Optional)

Customer records for the Customer Record Management System

ROW LEVEL SECURITY

Leave Row Level Security enabled.

Do not disable it.

Create the following columns.

Column Name

id

Type

uuid

Default Value

gen_random_uuid()

Primary Key

Yes

Column Name

user_id

Type

uuid

Allow NULL

No

Column Name

customer_type

Type

text

Allow NULL

No

Column Name

first_name

Type

text

Allow NULL

No

Column Name

last_name

Type

text

Allow NULL

No

Column Name

email

Type

text

Allow NULL

Yes

Column Name

phone

Type

text

Allow NULL

No

Column Name

company

Type

text

Allow NULL

Yes

Column Name

job_title

Type

text

Allow NULL

Yes

Column Name

address

Type

text

Allow NULL

Yes

Column Name

city

Type

text

Allow NULL

Yes

Column Name

state

Type

text

Allow NULL

Yes

Column Name

country

Type

text

Allow NULL

Yes

Column Name

notes

Type

text

Allow NULL

Yes

Column Name

created_at

Type

timestamp with time zone

Default Value

now()

Allow NULL

No

Column Name

updated_at

Type

timestamp with time zone

Default Value

now()

Allow NULL

No

Review every column carefully.

Then click:

Save

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

Open:

customers

inside the Table Editor.

Confirm that every column exists.

Verify:

☒ id is the Primary Key.

☒ user_id exists.

☒ customer_type exists.

☒ first_name exists.

☒ last_name exists.

☒ phone exists.

☒ email exists.

☒ company exists.

☒ job_title exists.

☒ address exists.

☒ city exists.

☒ state exists.

☒ country exists.

☒ notes exists.

☒ created_at exists.

☒ updated_at exists.

Also confirm that:

☒ Row Level Security remains enabled.

CHECKPOINT

Before moving on, confirm that:

☒ The customers table exists.

☒ Every required column exists.

☒ Every optional column exists.

☒ The Primary Key is correct.

☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

One common mistake is making the email field required.

In many real businesses, a customer may not have an email address available.

Phone remains the primary contact method in this workbook.

Another common mistake is forgetting the updated_at column.

Although it is not immediately required, it becomes useful later when records are edited and sorted.

BEHIND THE SCENES

Unlike the Expense Tracker, where each row represented one transaction, every row in this table represents one customer.

As the application grows, the software will retrieve, display and manage these customer records in different ways.

The structure you create today will support every feature built later.

THINK LIKE A SOFTWARE DESIGNER

A well-designed table should continue serving the application even after many new features are added.

Good database design anticipates future growth instead of solving only today's problem.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a richer database table
- distinguish required and optional fields
- prepare a customer database for future features
- keep Row Level Security enabled

In the next lesson, you will learn how to use database constraints to improve data quality before creating the Row Level Security policies.

CHAPTER 4**BUILDING THE CUSTOMER DATABASE****LESSON 3**

IMPROVING DATA QUALITY WITH DATABASE CONSTRAINTS

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In the previous lesson, you created the customers table.

In this lesson, you will improve the quality of the data that can be stored inside that table.

Not every piece of information entered by a user should automatically be accepted.

For example, the Customer Type field should contain only valid values.

This lesson introduces simple database constraints that help keep your customer records clean and consistent.

WHY THIS MATTERS

Imagine one customer record contains:

Business

Another contains:

business

Another contains:

Company

Another contains:

Corporate

Although they all mean almost the same thing, the software now has four different values for one idea.

That makes searching, filtering and reporting much more difficult.

Database constraints reduce this problem by limiting what can be stored.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The customers table exists.
- ☒ Every required column exists.
- ☒ Every optional column exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, review your customers table inside Supabase.

Confirm that:

`customer_type`

will always contain one of these values:

Individual

Business

Throughout the rest of this workbook, your application should always save one of these two values.

Do not use variations such as:

`individual`

`business`

`Company`

`Corporate`

`Person`

`Customer`

Keep the values consistent.

Next, review the remaining fields.

Confirm that:

first_name

last_name

phone

cannot be left empty.

Review every column one more time before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Review the customers table.

Confirm that:

☒ customer_type will always be:

Individual

or

Business

Confirm that:

☒ first_name is required.

☒ last_name is required.

☒ phone is required.

Confirm that:

☒ email remains optional.

☒ company remains optional.

☒ job_title remains optional.

☒ address remains optional.

☒ city remains optional.

☒ state remains optional.

☒ country remains optional.

☒ notes remains optional.

CHECKPOINT

Before moving on, confirm that you understand:

☒ Why consistent values matter.

☒ Which fields are required.

☒ Which fields are optional.

COMMON BEGINNER MISTAKES

A common mistake is allowing several different words to represent the same idea.

For example:

Business

business

BUSINESS

Company

Corporate

Keeping one standard value makes future searches, filters and reports much easier to build.

BEHIND THE SCENES

Your application will later display filters based on `customer_type`.

If every record stores different values, those filters become unreliable.

Using consistent values from the beginning prevents unnecessary problems later.

THINK LIKE A SOFTWARE DESIGNER

Good software should encourage consistent information.

Small decisions made early in a project often make future development much simpler.

WHAT YOU LEARNED

In this lesson you learned how to:

- improve data quality
- use consistent values
- prepare the database for reliable searching and filtering

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 4

ENABLING ROW LEVEL SECURITY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will confirm that Row Level Security is enabled for the customers table.

This is one of the most important security features in the entire application.

WHY THIS MATTERS

Every authenticated user will store customer records inside the same database table.

Without Row Level Security, users could potentially access customer records that belong to someone else.

Row Level Security prevents this.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The customers table exists.
- ☒ Authentication is working.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open your Supabase project.

Open:

Table Editor

Select:
customers
Confirm that:
Row Level Security
is enabled.
If it is already enabled, do not change anything.
Do not disable it.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Confirm that:
☒ Row Level Security is enabled.

CHECKPOINT

Before moving on, confirm that:
☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

Some beginners disable Row Level Security while trying to solve another problem.
Do not do this.
Throughout this workbook, Row Level Security should remain enabled.

BEHIND THE SCENES

Authentication identifies the current user.

Row Level Security decides what that user is allowed to access.

Both are required for secure software.

THINK LIKE A SOFTWARE DESIGNER

Security should be built into the application from the beginning.

It should never become an afterthought.

WHAT YOU LEARNED

In this lesson you learned how to:

- confirm Row Level Security
- understand its relationship with authentication
- prepare the database for secure customer records

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 5

CREATING THE SELECT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the SELECT policy for the customers table.

This policy controls which customer records a user is allowed to view.

WHY THIS MATTERS

Without a SELECT policy, users cannot safely retrieve customer records.

The policy ensures that every user sees only their own customers.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The customers table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open:

Authentication

Then:

Policies

Locate the:

customers

table.

Create a new policy.

Choose:

SELECT

Policy Name

Users can view only their own customers

Use this policy expression exactly:

`user_id = auth.uid()`

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

☒ The customers table now contains one SELECT policy.

Verify that:

☒ The policy applies to SELECT.

☒ The policy expression is:

```
user_id = auth.uid()
```

CHECKPOINT

Before moving on, confirm that:

- ✓ The SELECT policy exists.
- ✓ The policy has been saved.
- ✓ The policy compares:

```
user_id
```

```
with
```

```
auth.uid()
```

COMMON BEGINNER MISTAKES

A common mistake is comparing the wrong field.

Always compare:

```
user_id
```

```
with:
```

```
auth.uid()
```

No other comparison should be used.

BEHIND THE SCENES

Whenever your application requests customer records, Supabase checks this policy before returning any data.

Only rows belonging to the authenticated user will be returned.

THINK LIKE A SOFTWARE DESIGNER

Security should happen automatically.

Your application should not have to remember to protect every request.

The database should enforce those rules itself.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a SELECT policy
- protect customer records
- allow users to view only their own information

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 6

CREATING THE INSERT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the INSERT policy.

This policy controls which new customer records may be saved.

WHY THIS MATTERS

Every new customer should belong to the authenticated user who created it.

The INSERT policy ensures that only authorised records can be stored.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open:

Authentication

Then:

Policies

Locate:

customers

Create a new policy.

Choose:

INSERT

Policy Name

Users can insert only their own customers

Use this policy expression exactly:

`user_id = auth.uid()`

Review the policy.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that your customers table now contains:

☒ SELECT

☒ INSERT

Open the INSERT policy.

Verify:

☒ The policy applies to INSERT.

☒ The expression is:

```
user_id = auth.uid()
```

CHECKPOINT

Before moving on, confirm that:

☒ The INSERT policy exists.

☒ The policy has been saved successfully.

COMMON BEGINNER MISTAKES

Some beginners believe that authentication alone allows records to be inserted.

Authentication identifies the user.

The INSERT policy gives permission to save the record.

Both are required.

BEHIND THE SCENES

Whenever your application creates a new customer, Supabase checks this policy before saving the record.

If the customer belongs to the authenticated user, the record is accepted.

Otherwise, it is rejected.

THINK LIKE A SOFTWARE DESIGNER

Every database action should have its own security rule.

Viewing data and saving data are two different permissions.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an INSERT policy

- protect new customer records
- understand how authentication and Row Level Security work together

In the next lessons, you will create the UPDATE and DELETE policies before carrying out a complete security audit of the customer database.

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 7

CREATING THE UPDATE POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the UPDATE policy for the customers table.

This policy controls which customer records users are allowed to modify.

Once this lesson is complete, authenticated users will be able to update only the customer records they own.

WHY THIS MATTERS

Customer information changes over time.

A customer may:

- Change their phone number.
- Change their email address.
- Move to another city.
- Start working for a different company.
- Receive additional notes.

Your application should allow those changes while ensuring users cannot modify another person's customer records.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The SELECT policy exists.

☒ The INSERT policy exists.

☒ The customers table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the UPDATE policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

Authentication

Then click:

Policies

Locate the:

customers

table.

Create a new policy.

Choose:

UPDATE

Policy Name

Users can update only their own customers

Use the following policy expression exactly:

```
user_id = auth.uid()
```

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your customers table now contains:

☒ SELECT

☒ INSERT

☒ UPDATE

Open the UPDATE policy.

Verify that:

☒ The policy applies to UPDATE.

☒ The policy name is correct.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The UPDATE policy exists.

☒ The UPDATE policy has been saved.

☒ The policy compares:

```
user_id
```

```
with
```

```
auth.uid()
```

COMMON BEGINNER MISTAKES

Some beginners believe that identifying a customer record by its ID is enough when updating it.

It is not.

Later in this workbook, your JavaScript will update customer records by checking both:

- The customer ID
- The authenticated user's ID

The database will then apply the UPDATE policy as an additional layer of protection.

BEHIND THE SCENES

Every time someone edits a customer record, Supabase checks the UPDATE policy before saving the changes.

If the record belongs to the authenticated user, the update succeeds.

If it belongs to another user, the request is rejected automatically.

THINK LIKE A SOFTWARE DESIGNER

Reading information and changing information are different permissions.

Professional software treats them separately.

Just because someone can see a customer record does not automatically mean they should be allowed to change it.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an UPDATE policy
- protect customer records during editing

- understand why updates require their own permission

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 8

CREATING THE DELETE POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the final Row Level Security policy for your customers table.

This policy controls which customer records users are allowed to remove.

After completing this lesson, every important database action will have its own security policy.

WHY THIS MATTERS

Deleting a customer record permanently removes information from the database.

Your application must ensure that users can delete only the customer records they own.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ The INSERT policy exists.
- ☒ The UPDATE policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the DELETE policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

Authentication

Then click:

Policies

Locate the:

customers

table.

Create a new policy.

Choose:

DELETE

Policy Name

Users can delete only their own customers

Use the following policy expression exactly:

`user_id = auth.uid()`

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your customers table now contains:

☒ SELECT

☒ INSERT

☒ UPDATE

☒ DELETE

Open the DELETE policy.

Verify that:

☒ The policy applies to DELETE.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The DELETE policy exists.

☒ The DELETE policy has been saved successfully.

☒ The policy compares:

`user_id`

with

`auth.uid()`

COMMON BEGINNER MISTAKES

Some beginners believe that because a user created a customer record, they automatically have permission to delete it.

That permission comes from the DELETE policy.

Without it, the database rejects the request.

BEHIND THE SCENES

Whenever your application attempts to delete a customer record, Supabase checks the DELETE policy first.

If the customer belongs to the authenticated user, the record is removed.

If not, the request is rejected automatically.

THINK LIKE A SOFTWARE DESIGNER

Deleting information should always receive special attention.

Professional software protects permanent actions with multiple layers of security instead of assuming every request is valid.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a DELETE policy
- protect customer records from unauthorised deletion
- complete the four essential Row Level Security policies

CHAPTER 4

BUILDING THE CUSTOMER DATABASE

LESSON 9

TESTING THE CUSTOMER DATABASE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will verify that your customer database has been built correctly.

You are not creating new tables or policies.

Instead, you will review everything completed so far before customer management begins.

WHY THIS MATTERS

The customer database is the foundation of every feature that follows.

If there is a mistake in the database structure or security policies, adding, editing and deleting customers may not work correctly.

Professional developers review their database carefully before writing application features.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The customers table exists.
- ☒ Row Level Security remains enabled.
- ☒ All four policies have been created.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete the following review.

Open:

Table Editor

Open:

customers

Review every column.

Next, open:

Authentication

Policies

Review every policy.

Confirm that:

SELECT

uses:

user_id = auth.uid()

INSERT

uses:

user_id = auth.uid()

UPDATE

uses:

user_id = auth.uid()

DELETE

uses:

user_id = auth.uid()

Finally, confirm that Row Level Security is still enabled.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Complete this checklist.

Customers Table

- ☒ id
- ☒ user_id
- ☒ customer_type
- ☒ first_name
- ☒ last_name
- ☒ email
- ☒ phone
- ☒ company
- ☒ job_title
- ☒ address
- ☒ city
- ☒ state
- ☒ country
- ☒ notes
- ☒ created_at
- ☒ updated_at

Security

☒ Row Level Security enabled

Policies

☒ SELECT

☒ INSERT

☒ UPDATE

☒ DELETE

Every policy should compare:

user_id

with

auth.uid()

CHECKPOINT

Before moving on, confirm that:

- ☒ The customers table is complete.
- ☒ Every required column exists.
- ☒ Every policy exists.
- ☒ Row Level Security remains enabled.
- ☒ No policy has been removed accidentally.

COMMON BEGINNER MISTAKES

Some students accidentally delete a policy while experimenting inside Supabase.

Before moving to the next chapter, confirm that all four policies are still present.

Another common mistake is forgetting that every policy should use the same ownership check:

user_id = auth.uid()

Changing one policy while leaving the others unchanged can produce confusing behaviour later.

BEHIND THE SCENES

Your customer database is now fully prepared.

The next chapter will not change the database.

Instead, it will begin using this database to store and retrieve real customer information.

Because the database already protects every row, your application can focus on providing a good user experience while Supabase continues enforcing security behind the scenes.

THINK LIKE A SOFTWARE DESIGNER

A secure application begins with a secure database.

When the foundation is built correctly, every feature added later becomes easier to develop and easier to trust.

Professional developers invest time in getting the database right before writing business logic.

WHAT YOU LEARNED

In this lesson you learned how to:

- review a customer database
- verify Row Level Security
- verify every database policy
- confirm that the application is ready for customer management

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now has a secure database capable of storing complete customer profiles.

You designed a richer data model than the one used in Workbook 01 and created fields suitable for real business use.

You also enabled Row Level Security and created separate SELECT, INSERT, UPDATE and DELETE policies to protect every customer record.

Every authenticated user will be able to:

- ☒ View only their own customers.
- ☒ Create only their own customers.
- ☒ Update only their own customers.
- ☒ Delete only their own customers.

The database will automatically reject requests that do not satisfy these security rules.

CHAPTER MILESTONE

By the end of Chapter 4, you have successfully built:

- ✓ customers table
- ✓ Rich customer data model
- ✓ Required and optional fields
- ✓ Primary Key
- ✓ User ownership
- ✓ Row Level Security
- ✓ SELECT policy
- ✓ INSERT policy
- ✓ UPDATE policy
- ✓ DELETE policy
- ✓ Secure database foundation

TRANSITION TO CHAPTER 5

Your Customer Record Management System now has everything it needs behind the scenes.

Users can create accounts, sign in securely, and the database is ready to store customer information.

The next step is to build the customer dashboard.

You will create a professional customer entry form, introduce richer business forms, detect duplicate customer records before saving, and begin building business-oriented dashboard statistics.

This chapter introduces several new software design patterns that you have not used before and marks a significant step forward from Workbook 01.

CHAPTER 5

BUILDING THE CUSTOMER DASHBOARD

CHAPTER INTRODUCTION

Your Customer Record Management System now has a secure dashboard where authenticated users can sign in and manage their own customer information.

At the moment, however, the dashboard cannot do any meaningful work. There is no way to enter customer details, save them or measure how the business is growing.

That changes in this chapter.

You will build the complete customer creation workflow used by many real business applications.

By the end of this chapter, users will be able to enter customer information through a professional form, save customer records securely into Supabase, and immediately see useful business statistics update on the dashboard.

Although the application will still be relatively simple, it will already behave like real customer management software used by many organisations.

More importantly, you will continue learning how to use AI to build complete software features instead of isolated pieces of code.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A professional customer dashboard
- A rich customer entry form
- Customer information validation
- Customer information normalisation
- Duplicate email detection

- Duplicate phone number detection
- Secure customer saving
- Automatic form reset
- Business dashboard statistics
- Automatic dashboard refresh

By the end of the chapter, users will be able to create customer records confidently while the application helps maintain accurate, high-quality business information.

LESSON 1

DESIGNING THE CUSTOMER DASHBOARD

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will design the main working area of your Customer Record Management System.

This dashboard is where users will spend most of their time after signing in.

Rather than displaying a blank page, the dashboard will present useful business information alongside an organised customer management area.

The customer form and business statistics you build later in this chapter will fit naturally into this layout.

WHY THIS MATTERS

Business software is used repeatedly throughout the working day.

A good dashboard helps users understand where they are, what they can do next and how the business is performing.

A poorly organised dashboard slows people down.

Before adding customer management features, it is worth investing time in creating a clean, professional workspace.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works correctly.
- ☒ Email verification works.
- ☒ Login works.
- ☒ Dashboard authentication protection works.
- ☒ Logout works.

Only authenticated users should be able to reach the dashboard.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Transform the protected dashboard into a clean, professional customer management workspace.

Do not build customer saving yet.

Do not connect the customer form to Supabase yet.

This lesson focuses only on the dashboard layout and user experience.

DASHBOARD.HTML

Keep the existing authentication behaviour.

Retain:

- User email display
- Logout button

Create a professional dashboard containing the following sections.

Dashboard Header

Display:

- Page title:

Customer Dashboard

- Welcome message for the authenticated user
- Logout button

Business Statistics Section

Create four professional summary cards.

Each card should contain:

- Large number
- Descriptive label

Prepare cards for:

- Total Customers
- Customers Added Today
- New Customers This Month
- Business Customers

Use placeholder values of:

0

Give every statistic a unique ID so JavaScript can update them later.

Customer Management Section

Create a clearly separated section titled:

Add New Customer

Under this heading, prepare space for a professional customer form.

Do not connect the form to JavaScript yet.

Include fields for:

- Customer Type
- First Name
- Last Name
- Email
- Phone
- Company

- Job Title

- Address

- City

- State

- Country

- Notes

Include:

Add Customer

button.

Give every field a clear ID.

Customer Directory Section

Below the customer form, create another section titled:

Customer Directory

Display a simple placeholder message:

Your customer records will appear here after you begin adding customers.

Give this section a clear container ID for future lessons.

Status Message Area

Create one status message area above the customer form.

This area will later display:

- Success messages

- Validation messages

- Loading messages

Hide it initially.

DASHBOARD.JS

Keep:

- Authentication protection

- Logout

- Authenticated user's email display

Add safe references for:

- Customer form
- Status message
- Dashboard statistics
- Customer Directory

Do not:

- Save customers
- Validate fields
- Connect to Supabase
- Display customer cards

The page should load without JavaScript errors.

AUTH.CSS

Keep every existing style.

Design a modern business dashboard.

Style:

- Dashboard header
- Summary cards
- Customer form container
- Customer Directory container
- Section headings
- Form labels
- Input fields
- Text area
- Add Customer button
- Status message area

The design should feel clean and spacious.

Use generous spacing.

Keep the layout fully responsive.

On smaller screens:

- Stack dashboard cards neatly.
- Keep the customer form comfortable to complete.
- Prevent horizontal scrolling.

Do not introduce unnecessary visual effects.

IMPORTANT

Everything already built must continue working.

Do not build customer-saving logic.

Do not build validation.

Do not build dashboard statistics.

Do not retrieve customer records.

Those features belong to later lessons.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now provide a professional workspace ready for customer management.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Sign in.

Open the dashboard.

Confirm that:

- ☒ The dashboard header appears.
- ☒ The welcome message displays correctly.
- ☒ Four business statistic cards appear.
- ☒ The Add New Customer section appears.
- ☒ Every customer field appears.
- ☒ The Add Customer button appears.
- ☒ The Customer Directory placeholder appears.
- ☒ Logout still works.

Resize the browser window.

Confirm that:

- ☒ Dashboard cards remain organised.
- ☒ The customer form remains readable.
- ☒ No unnecessary horizontal scrolling appears.

CHECKPOINT

Before moving on, confirm that:

- ✓ The dashboard layout is complete.
- ✓ All customer fields are visible.
- ✓ The dashboard remains protected.
- ✓ Existing authentication continues working.
- ✓ Every future section has a clear placeholder.

COMMON BEGINNER MISTAKES

Some beginners immediately begin connecting forms to the database before finishing the dashboard layout.

Doing so makes it much harder to recognise whether a problem comes from the design or the JavaScript.

Complete the structure first.

Then add behaviour.

BEHIND THE SCENES

Professional software is usually developed in layers.

The first layer provides the visual structure.

The next layer adds behaviour.

The final layer connects the interface to the database.

Separating those responsibilities makes the software easier to build and maintain.

THINK LIKE A SOFTWARE DESIGNER

A good dashboard should answer three questions immediately:

Where am I?

What can I do here?

What information is most important?

Design every dashboard with those questions in mind.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports designing the customer dashboard. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional business dashboard
- organise information into logical sections
- prepare a dashboard for future functionality
- design software before connecting it to a database

CHAPTER 5

BUILDING THE CUSTOMER DASHBOARD

LESSON 2

BUILDING THE CUSTOMER ENTRY FORM

Estimated Time

70 minutes

WHAT YOU ARE BUILDING

In the previous lesson, you designed the Customer Dashboard and prepared space for the customer form.

In this lesson, you will build a professional customer entry form that businesses can use to collect customer information consistently.

The form will validate user input, clean customer information automatically, and prepare every customer record for secure storage.

The information will not be saved into Supabase yet.

Saving customer records will be completed in the next lesson.

WHY THIS MATTERS

Good customer records begin with good data entry.

If users can enter incomplete or inconsistent information, the customer database quickly becomes difficult to search and maintain.

A professional customer form helps prevent those problems before information reaches the database.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The Customer Dashboard appears correctly.
- ☒ All dashboard sections are visible.
- ☒ The Customer Directory placeholder appears.

✓ Dashboard authentication still works.

✓ Logout still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Build a professional customer entry form that validates and normalises customer information.

Do not save customer records into Supabase yet.

DASHBOARD.HTML

Keep everything already built.

Retain:

- Dashboard header
- Business statistics cards
- Customer Directory section
- Status message area

Complete the customer form.

Include:

- Customer Type
- First Name
- Last Name
- Email
- Phone
- Company
- Job Title
- Address
- City
- State
- Country
- Notes

Required fields:

- Customer Type
- First Name
- Last Name
- Phone

Optional fields:

- Email
- Company
- Job Title
- Address
- City
- State
- Country
- Notes

Use appropriate HTML controls.

Use descriptive labels.

Provide placeholder text where appropriate.

Associate every label with its input.

Give every field a clear ID.

Keep the Add Customer button.

DASHBOARD.JS

Keep every existing feature working.

Continue supporting:

- Authentication
- Logout
- Dashboard loading
- Authenticated user email

Implement professional form validation.

REQUIRED FIELDS

Before allowing submission:

Confirm that all required fields contain values.

Display friendly validation messages inside the status message area.

Do not use browser alert boxes.

EMAIL VALIDATION

If an email address has been entered:

Check that it follows a reasonable email format.

If invalid:

Display:

Enter a valid email address.

PHONE VALIDATION

Confirm that the phone number contains a reasonable number of digits.

Reject obviously invalid numbers.

NORMALISE CUSTOMER INFORMATION

Before validation succeeds:

FIRST NAME

- Remove unnecessary spaces.
- Replace repeated spaces with one space.
- Capitalise every word.

LAST NAME

Apply the same rules.

EMAIL

If entered:

- Trim spaces.
- Convert to lowercase.

PHONE

Remove:

- Spaces
- Hyphens
- Brackets

Preserve an existing leading plus sign where appropriate.

COMPANY

Trim spaces.

Replace repeated spaces with one space.

JOB TITLE

Apply the same rule.

ADDRESS

Trim unnecessary spaces.

Leave normal sentence formatting unchanged.

CITY

Trim spaces.

Capitalise every word.

STATE

Apply the same rule.

COUNTRY

Apply the same rule.

NOTES

Trim unnecessary spaces.

Leave the user's wording unchanged.

Write the cleaned values back into the visible form fields.

SUCCESSFUL VALIDATION

When every field passes validation:

Display:

Customer information has been prepared successfully and is ready to be saved.

Do not save the customer yet.

Do not contact Supabase.

AUTH.CSS

Keep every existing style.

Improve the customer form.

Style:

- Labels
- Inputs
- Select fields
- Text area
- Required field indicators
- Validation messages
- Success messages
- Form spacing
- Responsive layout

Keep the design clean and spacious.

IMPORTANT

Everything already built must continue working.

Do not save customer records.

Do not perform duplicate detection.

Do not calculate dashboard statistics.

Those features belong to the next lesson.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The customer form should now validate and normalise customer information professionally.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Open the dashboard.

Enter:

First Name

john

Last Name

smith

Email

JOHN@GMAIL.COM

Phone

0803 123 4567

Company

ABC LIMITED

City

lagos

State

lagos

Select:

Add Customer

Confirm that:

☒ First Name becomes:

John

☒ Last Name becomes:

Smith

☒ Email becomes lowercase.

☒ Phone is cleaned.

☒ Company spacing is corrected.

☒ City becomes:

Lagos

☒ State becomes:

Lagos

The page should display:

Customer information has been prepared successfully and is ready to be saved.

CHECKPOINT

Before moving on, confirm that:

- ✓ Validation works.
- ✓ Required fields work.
- ✓ Email validation works.
- ✓ Phone validation works.
- ✓ Customer information is normalised.
- ✓ No customer record has been saved yet.

COMMON BEGINNER MISTAKES

Many beginners validate customer information after saving it.

Professional software validates and cleans information before it ever reaches the database.

Doing so produces more consistent customer records and reduces future problems.

BEHIND THE SCENES

Your customer form now behaves like forms used in many professional business applications.

Instead of trusting users to enter perfectly formatted information, the software improves the quality of the data automatically before it can be stored.

THINK LIKE A SOFTWARE DESIGNER

Every customer added today may still exist in the business database years from now.

Investing in clean data entry now saves countless hours of correction later.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports building the customer entry form. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional customer form
- validate customer information
- normalise customer data
- prepare information for secure storage

CHAPTER 5

BUILDING THE CUSTOMER DASHBOARD

LESSON 3

SAVING CUSTOMER RECORDS SECURELY

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

Your customer form can now collect high-quality information.

In this lesson, you will complete the entire customer creation workflow.

The application will check for duplicate customer records, save authorised customer information into Supabase, reset the form, and immediately refresh the business dashboard.

By the end of this lesson, your Customer Record Management System will be capable of securely creating new customer records just like a real business application.

WHY THIS MATTERS

Creating customer records is one of the most important responsibilities of this software.

The workflow must protect data quality, maintain customer privacy, and ensure that every record belongs only to the authenticated user.

Once this lesson is complete, businesses will be able to trust the customer information stored inside the application.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer form validation works.
- ☒ Customer information is normalised.
- ☒ The customers table exists.
- ☒ Row Level Security remains enabled.

- ✓ INSERT policy exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Complete the customer creation workflow.

The application should:

- Check duplicate email addresses.
- Check duplicate phone numbers.
- Save customer records securely.
- Reset the form.
- Refresh dashboard statistics automatically.

DASHBOARD.JS

Keep every existing feature working.

Continue supporting:

- Authentication

- Dashboard
 - Customer form
 - Validation
 - Normalisation
-

AUTHENTICATED USER

Before saving:

Retrieve the authenticated user.

If authentication cannot be confirmed:

Redirect immediately to:

login.html

DUPLICATE EMAIL

If an email address has been entered:

Search only customer records belonging to the authenticated user.

If another customer already uses that email:

Display:

A customer with this email address already exists.

Do not continue.

DUPLICATE PHONE

Search only customer records belonging to the authenticated user.

If another customer already uses the same phone number:

Display:

A customer with this phone number already exists.

Do not continue.

SAVE CUSTOMER

When all checks succeed:

Insert one new customer into:

customers

Insert:

- user_id
- customer_type
- first_name
- last_name
- email
- phone
- company
- job_title
- address
- city
- state
- country
- notes

Do not insert:

created_at

updated_at

Allow Supabase to generate them automatically.

BUTTON BEHAVIOUR

While saving:

Disable:

Add Customer

Display a saving message.

When saving finishes:

Enable the button again.

SUCCESS

Display:

Customer saved successfully.

Clear the complete customer form.

Return Customer Type to its default value.

Leave the dashboard ready for another customer.

ERROR HANDLING

Handle unexpected errors professionally.

Display friendly messages.

Record technical information using:

`console.error()`

Do not display raw database errors.

DASHBOARD REFRESH

After a successful save:

Refresh:

- Business statistics
- Customer Directory

Do not require the user to refresh the browser.

AUTH.CSS

Keep every existing style.

Improve styling for:

- Saving messages
- Success messages
- Error messages
- Disabled Add Customer button

IMPORTANT

- Everything already built must continue working.
- Continue using the shared supabaseClient.
- Do not create another Supabase client.
- Every customer must belong to the authenticated user.
- Continue relying on Row Level Security.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The Customer Record Management System should now support the complete customer creation workflow.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

- Before replacing an existing file, copy your complete project folder.
- Save the copy outside your working project folder.
- Name the backup clearly using the current chapter and lesson.
- Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in.

Open the dashboard.

Add a new customer.

Confirm that:

- ☒ Validation succeeds.
- ☒ Duplicate detection works.
- ☒ The customer saves successfully.
- ☒ The form clears.
- ☒ The Add Customer button becomes disabled while saving.
- ☒ The button becomes active afterwards.

Open:

Supabase

Table Editor

customers

Confirm that:

- ☒ The customer record exists.
- ☒ user_id belongs to the authenticated user.
- ☒ created_at exists.
- ☒ updated_at exists.

Attempt to create another customer using the same email address.

Confirm that the application prevents the duplicate.

Repeat using the same phone number.

CHECKPOINT

Before moving on, confirm that:

- ✓ Customer records save successfully.
- ✓ Duplicate email detection works.
- ✓ Duplicate phone detection works.
- ✓ The form resets correctly.
- ✓ Customer records belong to the authenticated user.
- ✓ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

- A common mistake is checking duplicate records across the entire database.
- Customer creation should compare only records belonging to the authenticated user.
- Another common mistake is trusting the browser to supply the user ID.
- Always obtain the authenticated user's ID directly from Supabase before creating the customer record.

BEHIND THE SCENES

- The customer creation workflow now follows the same sequence used in many professional business systems.
- Information is validated, cleaned, checked for duplicates, stored securely, and immediately reflected throughout the application.
- Each step builds on the previous one to maintain reliable business data.

THINK LIKE A SOFTWARE DESIGNER

- Creating a customer record is more than inserting information into a database.
- It is a carefully controlled business process that protects data quality, user privacy and future reporting.

CODE-READING QUESTION

- Open dashboard.js. Find one function that supports saving customer records securely. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- detect duplicate customers
- save customer records securely
- protect customer ownership
- refresh application data automatically
- complete a professional customer creation workflow

CHAPTER 5

BUILDING THE CUSTOMER DASHBOARD

LESSON 4

BUILDING BUSINESS DASHBOARD STATISTICS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

Your Customer Record Management System can now create customer records securely.

In this lesson, you will transform the dashboard from a simple data entry screen into a useful business dashboard.

The dashboard will automatically calculate and display meaningful customer statistics every time the page loads and whenever a new customer is added.

Instead of forcing users to count customer records themselves, the software will do the work automatically.

WHY THIS MATTERS

One of the biggest advantages of business software is its ability to turn raw information into useful insight.

Business owners should be able to answer important questions immediately, such as:

- How many customers do I have?
- How many customers did I add today?
- How many customers have I added this month?
- How many business customers do I currently manage?

Providing these answers automatically makes the application much more valuable.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer records save successfully.

- ✓ Duplicate detection works.
- ✓ The customer form clears after saving.
- ✓ The customers table contains several records.
- ✓ Dashboard authentication still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

- I already have a working Customer Record Management System.
- Everything built so far must continue working exactly as before.
- Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.
- Return complete updated files only.
- Do not return snippets or partial code.
- Update only:
- `dashboard.html`
 - `dashboard.js`
 - `auth.css`

GENERAL GOAL

- Calculate and display live customer statistics on the dashboard.
- The statistics should refresh automatically whenever the dashboard loads or a new customer is added.

DASHBOARD.HTML

- Keep everything already built.
- Retain:
- Dashboard header
 - Customer form

- Customer Directory placeholder

- Status message area

Keep the existing four dashboard statistic cards.

Use their existing IDs.

Do not create additional statistic cards.

DASHBOARD.JS

Keep every existing feature working.

Continue supporting:

- Authentication
- Customer saving
- Customer validation
- Customer normalisation
- Duplicate detection

LOAD CUSTOMER RECORDS

After authentication succeeds:

Retrieve all customer records belonging to the authenticated user.

Continue relying on Row Level Security.

Do not retrieve another user's customer records.

CALCULATE

Using the retrieved customer records, calculate:

Total Customers

Count every customer record.

Customers Added Today

Count customer records whose:

created_at

matches today's date.

New Customers This Month

Count customer records created during the current calendar month.

Business Customers

Count customer records where:

customer_type

equals:

Business

DISPLAY

Update the four dashboard statistic cards.

If no customer records exist:

Display:

0

for every statistic.

AUTOMATIC REFRESH

Refresh all four statistics:

- When the dashboard first loads.
- After every successful customer save.

Do not require the user to refresh the browser manually.

ERROR HANDLING

Handle unexpected errors professionally.

Display friendly messages where appropriate.

Record technical information using:

console.error()

AUTH.CSS

Keep every existing style.

Improve the appearance of the statistic cards.

Style:

- Large statistic values
- Statistic labels
- Responsive dashboard grid
- Consistent spacing

Maintain a clean business appearance.

IMPORTANT

Everything already built must continue working.

Do not retrieve another user's customer records.

Continue relying on Row Level Security.

Do not calculate statistics manually.

Always calculate them from the latest customer records.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should automatically display live customer statistics.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in.

Open the dashboard.

Confirm that:

- ☒ Total Customers displays correctly.
- ☒ Customers Added Today displays correctly.
- ☒ New Customers This Month displays correctly.
- ☒ Business Customers displays correctly.

Now add another Business customer.

Confirm that:

- ☒ Total Customers increases.
- ☒ Business Customers increases.
- ☒ Customers Added Today updates when appropriate.
- ☒ Dashboard statistics refresh automatically.

The browser should not require a manual refresh.

CHECKPOINT

Before moving on, confirm that:

- ✓ Dashboard statistics calculate correctly.
- ✓ Statistics refresh automatically.
- ✓ Customer creation still works.
- ✓ Duplicate detection still works.
- ✓ Only authenticated user's records are counted.

COMMON BEGINNER MISTAKES

Some beginners manually increase the displayed numbers after saving a customer.

Professional software recalculates statistics from the database.

This guarantees that the dashboard always reflects the actual customer records stored in the system.

BEHIND THE SCENES

The dashboard does not store separate statistics.

Instead, it calculates them from the customer records already stored in the database.

Whenever new information is added, the calculations simply run again using the latest data.

This keeps the dashboard accurate without storing duplicate information.

THINK LIKE A SOFTWARE DESIGNER

A dashboard should help users make decisions quickly.

Whenever information can be calculated automatically from existing data, let the software do the work instead of expecting users to calculate it themselves.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building business dashboard statistics. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- calculate business statistics
- display live dashboard information
- refresh statistics automatically
- improve business visibility through software

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System has now evolved from a protected dashboard into a working customer management application.

Users can enter customer information through a professional form, while the software validates the information, normalises it for consistency, prevents duplicate customer records, and stores everything securely inside Supabase.

The dashboard now provides meaningful business insight by automatically calculating customer statistics whenever records change.

At this stage, your application already demonstrates many of the same workflows found in commercial Customer Relationship Management (CRM) systems.

CHAPTER MILESTONE

By the end of Chapter 5, you have successfully built:

- ✓ Professional Customer Dashboard
- ✓ Rich Customer Entry Form
- ✓ Customer Information Validation
- ✓ Customer Information Normalisation
- ✓ Duplicate Email Detection
- ✓ Duplicate Phone Detection
- ✓ Secure Customer Creation
- ✓ Automatic Form Reset
- ✓ Live Business Dashboard Statistics
- ✓ Automatic Dashboard Refresh

TRANSITION TO CHAPTER 6

Your Customer Record Management System can now create customer records successfully.

The next challenge is helping users work with those records efficiently.

A business owner should not have to open Supabase every time they want to view customer information.

Instead, the dashboard should display every customer in a clean, organised directory.

In the next chapter, you will build a professional Customer Directory that displays customer cards, supports contact actions, handles loading and error states gracefully, and adapts beautifully to different screen sizes.

CHAPTER 6

VIEWING CUSTOMERS

CHAPTER INTRODUCTION

Customer information becomes valuable only when people can access it easily.

At the moment, your application stores customer records successfully, but those records remain hidden inside the database.

In this chapter, you will transform the dashboard into a practical customer management workspace by displaying every customer as a professional customer card.

Users will immediately see important customer information without leaving the application.

The directory will also provide useful actions such as calling customers, sending emails and opening detailed customer profiles.

By the end of this chapter, your Customer Record Management System will feel much closer to the type of software businesses use every day.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Professional Customer Directory
- Customer Profile Cards
- Customer Initials
- Click-to-call
- Click-to-email
- View Profile action
- Loading state
- Empty state

- Error state
- Responsive Customer Directory
- Customer privacy testing

LESSON 1

BUILDING THE CUSTOMER DIRECTORY

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete Customer Directory.

The application will retrieve customer records from Supabase, display them as professional customer cards, generate customer initials automatically, provide click-to-call and click-to-email actions, and allow users to open individual customer profiles.

The directory will also provide friendly loading, empty and error states to improve the overall user experience.

WHY THIS MATTERS

Businesses rarely interact directly with database tables.

Instead, they work with organised customer lists that present information clearly and make common tasks quick and convenient.

A well-designed customer directory allows users to recognise customers immediately and perform common actions with very little effort.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer records save successfully.
- ☒ Dashboard statistics work.
- ☒ Several customer records exist.
- ☒ The Customer Directory placeholder appears.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Replace the Customer Directory placeholder with a fully functional customer directory.

Retrieve customer records belonging only to the authenticated user and display them as professional customer cards.

Each card should include:

- Customer initials generated from the customer's first and last names.
- Full customer name.
- Customer Type.
- Company when available.
- Phone number.
- Email address when available.
- Click-to-call using `tel:.`
- Click-to-email using `mailto:.`
- View Profile button linking to:

`customer-profile.html?id=CUSTOMER_ID`

Use only the customer ID in the page address.

Do not include any other customer information in the URL.

Display customer records ordered by:

created_at

descending.

Create separate areas for:

- Loading customers...
- Empty customer directory
- Error loading customers

Only one state should be visible at a time.

If no customer records exist, display:

You haven't added any customers yet.

Add your first customer using the form above.

Handle unexpected errors professionally.

Record technical details using:

`console.error()`

Refresh the directory automatically whenever a customer is added.

Do not duplicate customer cards during refresh.

Continue relying on Row Level Security.

Style the directory using responsive customer cards that remain easy to use on mobile devices.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now display a complete Customer Directory.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in.

Confirm that:

- ☒ Customer cards appear.
- ☒ Initials display correctly.
- ☒ Phone numbers are clickable.
- ☒ Email addresses are clickable.
- ☒ View Profile buttons contain customer IDs.
- ☒ Loading state works.
- ☒ Empty state works.
- ☒ Customer cards refresh after adding another customer.
- ☒ No duplicate cards appear.
- ☒ Only authenticated user's customer records appear.

CHECKPOINT

Before moving on, confirm that:

- ✓ Customer Directory works correctly.
- ✓ Customer cards display correctly.
- ✓ Contact actions work.
- ✓ Loading, empty and error states work.
- ✓ Customer privacy remains protected.

COMMON BEGINNER MISTAKES

Some beginners retrieve every customer record from the database and then filter them inside JavaScript.

Always request only the authenticated user's records.

Doing so improves both security and performance.

BEHIND THE SCENES

Each customer card is created from one customer record returned by Supabase.

JavaScript converts database information into a visual component that users can recognise and interact with immediately.

THINK LIKE A SOFTWARE DESIGNER

Business software should present information in a way that reduces effort.

A well-designed customer directory allows users to recognise customers quickly and perform common actions without unnecessary navigation.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports building the customer directory. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional Customer Directory
- display customer profile cards
- generate customer initials
- add contact actions
- handle loading, empty and error states
- refresh displayed information automatically

CHAPTER 6

VIEWING CUSTOMERS

LESSON 2

BUILDING THE CUSTOMER PROFILE

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

The Customer Directory now allows users to see every customer at a glance.

In this lesson, you will build the complete Customer Profile page.

Instead of displaying only summary information, the profile page will securely retrieve one customer record from Supabase and display every available detail in a clean, professional layout.

Users will also be able to call customers, send emails, and begin the customer editing process from this page.

WHY THIS MATTERS

Customer cards are useful for browsing information quickly, but businesses often need to see the complete customer record before making decisions.

A dedicated profile page provides enough space to display contact information, addresses, business information, notes and record details without overcrowding the Customer Directory.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer Directory works correctly.
- ☒ View Profile buttons appear.
- ☒ Customer cards display correctly.
- ☒ Customer IDs are passed through the page address.
- ☒ customer-profile.html already exists.

- ✓ customer-profile.js already exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared supabaseClient from supabase-config.js.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- customer-profile.html
- customer-profile.js
- auth.css

GENERAL GOAL

Transform the protected customer profile page into a complete customer information page.

The page should:

- Read the customer ID from the page address.
- Validate that the ID exists.
- Confirm the user is authenticated.
- Retrieve only the authorised customer record.
- Display the complete customer profile.

CUSTOMER-PROFILE.HTML

Keep:

- Authentication protection
- Logout

Create a professional profile page.

Include:

Profile Header

Display:

- Customer initials
- Full customer name
- Customer Type

Business Information

Display:

- Company
- Job Title

Contact Information

Display:

- Phone
- Email

Provide:

- Call Customer
- Email Customer

actions.

Address

Display:

- Address
- City
- State
- Country

Customer Notes

Display customer notes.

If notes do not exist:

Display:

No notes have been added for this customer.

Record Information

Display:

- Date Added
- Last Updated

Profile Actions

Display:

- Edit Customer
- Back to Dashboard
- Logout

Page States

Create separate sections for:

- Loading customer profile
- Invalid customer ID
- Customer unavailable
- Profile content

Only one state should be visible at a time.

CUSTOMER-PROFILE.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js.

AUTHENTICATION

Confirm the user is authenticated.

If not:

Redirect immediately to:

login.html

READ CUSTOMER ID

Read:

id

from the page address using:

URLSearchParams.

Trim unnecessary spaces.

If no ID exists:

Display:

No customer was selected.

Return to the dashboard and choose a customer.

Do not contact Supabase.

RETRIEVE CUSTOMER

If an ID exists:

Retrieve one customer where:

- id matches the page address
- user_id matches the authenticated user

Continue relying on Row Level Security.

Use:

maybeSingle()

or an equivalent single-record query.

Do not retrieve every customer.

DISPLAY CUSTOMER

Display:

- Customer initials

- Full name

- Customer Type

- Company

- Job Title

- Phone

- Email

- Address

- City

- State

- Country

- Notes

- Date Added

- Last Updated

Generate customer initials automatically.

Format database dates into readable dates.

Display customer information safely.

Do not insert customer information as unsafe HTML.

CONTACT ACTIONS

Phone:

Create:

tel:

link.

Email:

Create:

mailto:

link when an email exists.

Do not create broken email links.

EDIT CUSTOMER

Configure:

Edit Customer

to open:

dashboard.html?edit=CUSTOMER_ID

Only pass the customer ID.

ERROR HANDLING

If no authorised customer is found:

Display:

This customer record could not be found or is unavailable.

Do not reveal ownership information.

Handle unexpected errors professionally.

Record technical information using:

console.error()

AUTH.CSS

Keep every existing style.

Design a professional customer profile page.

Style:

- Profile header
- Initials badge
- Information sections
- Contact actions
- Customer notes
- Record information
- Profile action buttons
- Loading state
- Invalid customer state
- Responsive profile layout

IMPORTANT

Everything already built must continue working.

Continue relying on Row Level Security.

Retrieve only one authorised customer.

Do not expose raw database errors.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- customer-profile.html
- customer-profile.js
- auth.css

The application should now support complete customer profiles.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open the Customer Directory.

Select:

View Profile

Confirm that:

- ☒ Correct customer profile appears.
- ☒ Initials display correctly.
- ☒ Contact information displays correctly.
- ☒ Call Customer works.
- ☒ Email Customer works.
- ☒ Edit Customer opens:

`dashboard.html?edit=CUSTOMER_ID`

- ☒ Dates display correctly.
- ☒ Missing notes display the default message.

Attempt to access another user's customer ID.

Confirm that:

- ☒ Customer information is not displayed.

CHECKPOINT

Before moving on, confirm that:

- ☒ Customer profile loads correctly.
- ☒ Authentication works.
- ☒ Customer ownership is protected.
- ☒ Contact actions work.
- ☒ Edit Customer works.

COMMON BEGINNER MISTAKES

Some beginners retrieve every customer record before selecting one inside JavaScript.

Always request only the customer that the page needs.

Doing so improves security, performance and keeps the application easier to maintain.

BEHIND THE SCENES

The customer profile page demonstrates one of the most common patterns in business software.

Instead of creating one HTML page for every customer, the application uses one reusable page that loads different information depending on the customer ID supplied.

THINK LIKE A SOFTWARE DESIGNER

Reusable pages make software easier to maintain.

Whenever many records share the same layout, build one dynamic page instead of creating many separate pages.

CODE-READING QUESTION

Open customer-profile.js. Find one function that supports building the customer profile. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete customer profile
- retrieve one authorised customer

- protect customer privacy
- create reusable profile pages
- build professional customer detail screens

CHAPTER 6

VIEWING CUSTOMERS

LESSON 3

TESTING THE CUSTOMER DIRECTORY

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will thoroughly test the complete Customer Directory and Customer Profile system.

Rather than adding new features, you will confirm that every part of the directory works correctly under different conditions.

Professional software development always includes careful testing before new features are added.

WHY THIS MATTERS

Many software problems appear only after several features begin working together.

Testing helps you discover those problems before users rely on the application.

This lesson will confirm that customer privacy, directory behaviour and profile pages all function correctly.

BEFORE YOU CONTINUE

Prepare:

- Two separate user accounts.
- Several Individual customers.
- Several Business customers.

Include customers with and without email addresses.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

Correct any problems before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

TEST 1

CUSTOMER DIRECTORY

Confirm that:

- ☒ Customer cards load.
- ☒ Loading state appears briefly.
- ☒ Empty state appears for new accounts.
- ☒ Error state behaves correctly.

TEST 2**CUSTOMER CARDS**

Confirm that every card displays:

- ☒ Customer initials
- ☒ Customer name
- ☒ Customer Type
- ☒ Company when available
- ☒ Phone
- ☒ Email
- ☒ View Profile

TEST 3**CONTACT ACTIONS**

Confirm that:

- ☒ Click-to-call works.
- ☒ Click-to-email works.
- ☒ Missing email addresses do not create broken links.

TEST 4**CUSTOMER PROFILE**

Open several customer profiles.

Confirm that:

- ☒ Correct information appears.
- ☒ Initials are correct.
- ☒ Dates display correctly.
- ☒ Customer notes display correctly.

TEST 5

EDIT CUSTOMER

Confirm that:

Edit Customer

opens:

dashboard.html?edit=CUSTOMER_ID

Verify that only the customer ID appears in the address.

TEST 6

ACCOUNT PRIVACY

Sign into another account.

Confirm that:

- ☒ Only that user's customers appear.
- ☒ Another user's customer ID cannot be opened.
- ☒ Customer statistics continue showing only the authenticated user's records.

TEST 7

RESPONSIVE DESIGN

Reduce the browser width.

Confirm that:

- ☒ Customer cards remain readable.
- ☒ Profile pages remain readable.
- ☒ Buttons remain usable.
- ☒ Horizontal scrolling does not appear.

CHECKPOINT

Before completing this chapter, confirm that:

- ☒ Customer Directory works correctly.
- ☒ Customer Profiles work correctly.
- ☒ Contact actions work.
- ☒ Authentication works.
- ☒ Customer privacy works.
- ☒ Responsive layouts work.

COMMON BEGINNER MISTAKES

Many beginners assume that software works because one successful test passes.

Professional software is tested under many different situations, including missing information, different users and unusual screen sizes.

BEHIND THE SCENES

Customer privacy depends on several layers working together:

- Authentication
- Secure queries
- Row Level Security
- Careful interface design

Removing any one of these layers weakens the application.

THINK LIKE A SOFTWARE DESIGNER

Good software is not only built carefully.

It is also tested carefully.

Professional developers spend significant time confirming that completed features continue working as new features are added.

WHAT YOU LEARNED

In this lesson you learned how to:

- test complete software features
- verify customer privacy
- verify responsive layouts
- test authentication
- confirm professional software quality

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now allows users to browse customer records through a professional Customer Directory and view complete customer profiles without leaving the application.

The software retrieves only authorised customer records, protects customer privacy, provides convenient contact actions and adapts smoothly to different screen sizes.

The application now resembles the type of customer management software used by many businesses every day.

CHAPTER MILESTONE

By the end of Chapter 6, you have successfully built:

- ✓ Customer Directory
- ✓ Customer Cards
- ✓ Customer Initials
- ✓ Click-to-call
- ✓ Click-to-email
- ✓ Customer Profile Page
- ✓ Edit Customer Action
- ✓ Loading State
- ✓ Empty State
- ✓ Error State
- ✓ Responsive Directory
- ✓ Customer Privacy Protection

TRANSITION TO CHAPTER 7

Your application can now create, organise and display customer information professionally.

The next step is making those customer records much easier to find.

As customer databases grow, scrolling through hundreds of customer cards quickly becomes inefficient.

In the next chapter, you will build intelligent search, filtering and sorting capabilities that allow businesses to locate customer records within seconds.

CHAPTER 7

SEARCHING, FILTERING AND SORTING CUSTOMERS

CHAPTER INTRODUCTION

Your Customer Record Management System can now create, store and display customer records professionally.

As the number of customers grows, however, finding the right customer becomes increasingly difficult.

Scrolling through hundreds of customer cards is slow and frustrating.

Professional customer management software solves this problem by giving users powerful tools for searching, filtering and organising information.

In this chapter, you will build a complete customer discovery system that allows users to locate customer records within seconds.

The Customer Directory will become far more useful without changing the customer information stored in the database.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Customer Directory toolbar
- Live customer search
- Search by name
- Search by company
- Search by phone number
- Search by email address
- Customer Type filtering

- Customer sorting
- Result counter
- No-results state
- Clear Search and Filters
- Automatic directory updates

LESSON 1

BUILDING CUSTOMER SEARCH, FILTERING AND SORTING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a complete Customer Directory toolbar.

Users will be able to:

- Search customers while typing.
- Filter customers by Customer Type.
- Sort customer records.
- Clear all search controls instantly.

The Customer Directory will update automatically without reloading the page.

WHY THIS MATTERS

Businesses often manage hundreds or thousands of customers.

Without search and filtering, finding the correct customer quickly becomes difficult.

Professional software should help users locate information in seconds instead of forcing them to browse long lists manually.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer Directory works correctly.
- ☒ Customer cards display correctly.
- ☒ Customer profiles work correctly.

- ✓ Dashboard statistics work.
- ✓ Several customer records exist.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Transform the Customer Directory into a searchable, filterable and sortable customer management system.

DASHBOARD.HTML

Keep everything already built.

Above the Customer Directory create a professional toolbar.

Include:

Search Customers

Label:

Search Customers

Placeholder:

Search by name, company, phone or email

Use:

type="search"

Customer Type

Label:

Customer Type

Options:

- All Customers
- Individual
- Business

Sort Customers

Label:

Sort Customers

Options:

- Newest First
- Oldest First
- Name A–Z
- Name Z–A

Clear Search and Filters

Create a button labelled:

Clear Search and Filters

Result Information

Create an area that will display messages such as:

Showing 18 of 24 customers

No Results

Create a hidden section displaying:

No customers match your current search or filters.

Try changing your search or clearing the filters.

This is different from the empty customer directory message.

DASHBOARD.JS

Keep every existing feature working.

Continue supporting:

- Authentication
 - Dashboard statistics
 - Customer creation
 - Customer Directory
 - Customer profiles
-

STORE CUSTOMER RECORDS

After customer records load successfully:

Store them inside a reusable array named:

allCustomers

or another clear equivalent.

Do not request customer records from Supabase every time the user searches.

SEARCH

Search while the user types.

Search across:

- First Name
- Last Name
- Full Name
- Company
- Phone
- Email

Ignore letter case.

Trim unnecessary spaces.

Phone searches should ignore:

- Spaces
- Hyphens
- Brackets

FILTER

Filter by:

Customer Type

Support:

- All Customers
- Individual
- Business

SORT

Support:

- Newest First
- Oldest First
- Name A–Z
- Name Z–A

Sort customer names using:

first_name

plus

last_name.

Do not permanently reorder:

allCustomers

Sort a copied array instead.

CLEAR SEARCH

Selecting:

Clear Search and Filters

should:

- Clear the search field.
- Return Customer Type to:

All Customers

- Return Sort Customers to:

Newest First

- Display every customer again.

RESULT INFORMATION

Update automatically.

Examples:

Showing 1 of 20 customer

Showing 7 of 20 customers

Use the correct singular and plural wording.

NO RESULTS

If customer records exist but none match:

Hide the customer cards.

Display:

No customers match your current search or filters.

Do not display the empty customer directory message.

AUTOMATIC REFRESH

After adding a customer:

Reload:

allCustomers

Reapply:

- Search

- Filter
- Sorting

Refresh the Customer Directory.

Do not create duplicate customer cards.

AUTH.CSS

Keep every existing style.

Style:

- Customer Directory toolbar
- Search field
- Customer Type selector
- Sort selector
- Clear button
- Result information
- No-results state

Keep the toolbar responsive.

Stack controls neatly on smaller screens.

IMPORTANT

Everything already built must continue working.

Do not send additional Supabase requests while the user types.

Continue using the existing customer rendering function.

Continue relying on:

allCustomers

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html

- dashboard.js
- auth.css

The Customer Directory should now support professional searching, filtering and sorting.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Confirm that:

- ☒ Searching by first name works.
- ☒ Searching by last name works.
- ☒ Searching by company works.
- ☒ Searching by phone works.
- ☒ Searching by email works.
- ☒ Customer Type filtering works.
- ☒ Sorting works.
- ☒ Clear Search and Filters works.
- ☒ Result information updates.
- ☒ No-results state appears correctly.
- ☒ Customer cards refresh automatically after adding another customer.

CHECKPOINT

Before moving on, confirm that:

- ✓ Search works.
- ✓ Filter works.
- ✓ Sorting works.
- ✓ No-results state works.
- ✓ Result information works.
- ✓ Directory refresh works.

COMMON BEGINNER MISTAKES

- Many beginners request customer records from the database every time the user presses a key.
- The application already has the customer information.
- Searching the existing customer array is much faster and provides a smoother user experience.

BEHIND THE SCENES

- The Customer Directory now behaves much like professional CRM software.
- The customer records are retrieved once.
- JavaScript then performs searching, filtering and sorting locally inside the browser, making the interface feel much faster.

THINK LIKE A SOFTWARE DESIGNER

- Good software does not make users adapt to the data.
- Instead, it adapts the data to the way users naturally search for information.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building customer search, filtering and sorting. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build live customer search
- filter customer records
- sort customer records
- improve software performance
- create a professional customer discovery experience

CHAPTER 7

SEARCHING, FILTERING AND SORTING CUSTOMERS

LESSON 2

TESTING CUSTOMER DISCOVERY

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete customer discovery system.

Rather than adding new functionality, you will confirm that searching, filtering and sorting continue working correctly under different situations.

WHY THIS MATTERS

Search features often work correctly when tested individually but fail when several controls are combined.

Professional testing ensures that every control behaves predictably.

BEFORE YOU CONTINUE

Prepare:

- Several Individual customers
- Several Business customers
- Customers with different names
- Customers with different creation dates

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Confirm that:

- ☒ Searching by first name works.
- ☒ Searching by last name works.
- ☒ Searching by company works.
- ☒ Searching by phone number works.
- ☒ Searching by email address works.
- ☒ Customer Type filtering works.
- ☒ Sorting by newest works.
- ☒ Sorting by oldest works.
- ☒ Name A–Z works.
- ☒ Name Z–A works.
- ☒ Search and filtering work together.
- ☒ Search and sorting work together.
- ☒ Filter and sorting work together.
- ☒ Clear Search and Filters restores the default directory.
- ☒ No-results state appears correctly.
- ☒ Empty directory state still works.
- ☒ Customer Directory remains responsive.

- ✓ Keyboard navigation works correctly.
- ✓ Adding a new customer refreshes the directory correctly.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Every Customer Directory control works.
- ✓ Combined controls work correctly.
- ✓ Customer records remain protected.
- ✓ Customer cards never duplicate.

COMMON BEGINNER MISTAKES

Some developers test every control separately but never test them together.

Professional software should behave correctly regardless of how many controls the user combines.

BEHIND THE SCENES

Every Customer Directory update follows the same sequence:

Retrieve customer records once.

Apply search.

Apply filtering.

Apply sorting.

Display the final result.

Using one predictable process makes the application easier to maintain.

THINK LIKE A SOFTWARE DESIGNER

When several controls affect the same information, build one central workflow that coordinates them instead of allowing each control to work independently.

WHAT YOU LEARNED

In this lesson you learned how to:

- test combined software features
- verify customer discovery
- test responsive interfaces
- validate professional application behaviour

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now allows users to locate customer information quickly using professional search, filtering and sorting tools.

Instead of browsing long customer lists manually, users can now find the information they need within seconds.

These improvements significantly increase the usability of the software while keeping customer information secure.

CHAPTER MILESTONE

By the end of Chapter 7, you have successfully built:

- ✓ Customer Directory Toolbar
- ✓ Live Customer Search
- ✓ Customer Type Filtering
- ✓ Customer Sorting
- ✓ Result Counter
- ✓ No-results State
- ✓ Clear Search and Filters
- ✓ Automatic Directory Refresh
- ✓ Professional Customer Discovery

TRANSITION TO CHAPTER 8

Your Customer Record Management System can now create, display and locate customer records efficiently.

The next step is allowing businesses to keep customer information accurate as it changes.

Customers move to new addresses.

Phone numbers change.

Companies grow.

Notes need updating.

In the next chapter, you will build a complete customer editing workflow that allows authorised users to update customer information securely while preserving data quality and customer privacy.

CHAPTER 8

EDITING CUSTOMER RECORDS

CHAPTER INTRODUCTION

Customer information rarely remains unchanged.

A customer may change their phone number, move to a new address, join a different company or update their email address.

Businesses also discover mistakes that need to be corrected after customer records have been created.

A professional Customer Record Management System must therefore allow authorised users to update customer information safely.

Editing customer records introduces an important challenge.

The application must identify exactly which customer is being edited, retrieve that customer's information securely, prevent duplicate customer details and ensure that one user can never modify another user's records.

In this chapter, you will build a complete customer editing workflow that follows the same security principles used throughout this workbook.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Edit Customer workflow
- Customer edit mode
- Automatic form population
- Secure customer retrieval for editing
- Customer information updating
- Validation during editing

- Customer information normalisation
- Duplicate detection during editing
- Secure database updates
- Cancel editing
- Automatic dashboard refresh
- Customer editing security tests

LESSON 1

BUILDING CUSTOMER EDITING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete customer editing workflow.

Users will be able to open a customer profile, select Edit Customer, automatically load the existing customer information into the dashboard form, update the information and save the changes securely.

The application will reuse the existing customer form instead of creating a separate editing form.

WHY THIS MATTERS

Professional software avoids unnecessary duplication.

Instead of maintaining one form for creating customers and another for editing customers, the same form can perform both tasks.

This reduces maintenance, keeps the user experience consistent and ensures that customer information always passes through the same validation rules.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer profiles work correctly.
- ☒ Edit Customer opens:
dashboard.html?edit=CUSTOMER_ID
- ☒ Customer creation still works.
- ☒ Customer Directory works.
- ☒ Dashboard statistics work.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `customer-profile.js`
- `auth.css`

GENERAL GOAL

Build the complete customer editing workflow using the existing customer form.

CUSTOMER-PROFILE.JS

Keep every existing feature working.

Configure:

Edit Customer

to open:

`dashboard.html?edit=CUSTOMER_ID`

Pass only the customer ID.

DASHBOARD.HTML

Keep the existing customer form.

Do not create another form.

Add:

- Form heading
- Form description
- Cancel Edit button

Keep:

- Add Customer button

The button text should change automatically during edit mode.

DASHBOARD.JS

Keep every existing feature working.

Continue supporting:

- Authentication
- Customer creation
- Dashboard statistics
- Customer Directory
- Customer search
- Customer profiles

EDIT MODE

Read:

edit

from the page address.

If no edit value exists:

Continue operating in:

Add Customer

mode.

If an edit value exists:

Switch to:

Edit Customer

mode.

Update:

- Form heading
- Form description
- Button text

Display:

Cancel Edit

LOAD CUSTOMER

When edit mode begins:

Retrieve one customer where:

- id equals the customer ID
- user_id equals the authenticated user

Continue relying on Row Level Security.

Do not retrieve every customer.

Populate the existing customer form with:

- Customer Type
- First Name
- Last Name
- Email
- Phone
- Company
- Job Title
- Address
- City
- State
- Country
- Notes

Handle missing optional information gracefully.

Display:

Customer information loaded successfully.

VALIDATION

Reuse the existing validation rules.

Do not create a second validation system.

Apply the same:

- Required field validation
- Email validation
- Phone validation
- Customer information normalisation

DUPLICATE DETECTION

Before updating:

Check duplicate email addresses.

Check duplicate phone numbers.

Exclude the customer currently being edited.

Search only the authenticated user's customer records.

UPDATE CUSTOMER

When validation succeeds:

Update:

customers

where:

- id equals the current customer ID
- user_id equals the authenticated user

Update:

- customer_type
- first_name
- last_name
- email
- phone

- company
- job_title
- address
- city
- state
- country
- notes

Update:

updated_at

using the current timestamp.

Do not modify:

created_at

SUCCESS

Display:

Customer updated successfully.

Return the form to:

Add Customer

mode.

Clear the form.

Refresh:

- Dashboard statistics
- Customer Directory

If the customer profile page is reopened, the updated information should appear.

CANCEL EDIT

Selecting:

Cancel Edit

should:

- Return the dashboard to Add Customer mode.
- Clear the customer form.

- Remove the:
edit
value from the page address.

AUTH.CSS

Keep every existing style.

Improve styling for:

- Edit mode
- Cancel Edit button
- Success messages
- Error messages
- Form status
- Responsive action buttons

IMPORTANT

Everything already built must continue working.

Continue relying on Row Level Security.

Use the existing customer form.

Do not create duplicate customer records during editing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- customer-profile.js
- auth.css

The application should now support complete customer editing.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open:

Customer Profile

Select:

Edit Customer

Confirm that:

- ☒ Edit mode begins.
- ☒ Customer information loads.
- ☒ Form heading changes.
- ☒ Button text changes.

Modify:

- Phone
- Company
- Notes

Select:

Save Changes

Confirm that:

- ☒ Validation works.
- ☒ Duplicate detection works.
- ☒ Customer updates successfully.
- ☒ Dashboard statistics refresh.
- ☒ Customer Directory refreshes.

Open the profile again.

Confirm that the updated information appears.

CHECKPOINT

Before moving on, confirm that:

- ☒ Edit mode works.
- ☒ Customer information loads.
- ☒ Validation still works.
- ☒ Duplicate detection still works.
- ☒ Customer updates successfully.
- ☒ Dashboard refreshes.

COMMON BEGINNER MISTAKES

Some beginners build a completely separate editing form.

Professional applications often reuse the same form for both creating and editing records.

This reduces duplicated code and keeps behaviour consistent.

BEHIND THE SCENES

The customer form now supports two modes.

When no customer ID exists, it creates a new customer.

When an edit customer ID exists, it updates the existing customer.

The same interface now supports two different business workflows.

THINK LIKE A SOFTWARE DESIGNER

Whenever two features perform nearly the same task, look for opportunities to reuse existing components instead of creating new ones.

Reusable software is easier to maintain and usually contains fewer bugs.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports building customer editing. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build complete customer editing
- reuse existing software components
- update customer records securely
- protect customer ownership
- refresh business information automatically

CHAPTER 8

EDITING CUSTOMER RECORDS

LESSON 2

TESTING CUSTOMER EDITING

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete customer editing workflow.

You will verify that customer updates remain secure, duplicate detection works correctly and customer information cannot be modified across user accounts.

WHY THIS MATTERS

Editing changes existing business information.

Testing helps ensure that authorised users can update their own customer records without accidentally affecting another user's data.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts.
- Several customer records.

Include duplicate email and phone number scenarios.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Confirm that:

- ☒ Edit Customer opens correctly.
- ☒ Customer information loads.
- ☒ Validation works.
- ☒ Normalisation works.
- ☒ Duplicate email detection works.
- ☒ Duplicate phone detection works.
- ☒ Existing customer email addresses are not treated as duplicates.
- ☒ Existing customer phone numbers are not treated as duplicates.
- ☒ Customer updates successfully.
- ☒ Dashboard statistics refresh.
- ☒ Customer Directory refreshes.
- ☒ Updated profile displays correctly.

Attempt to edit another user's customer.

Confirm that:

- ☒ Access is denied.
- ☒ Customer information is not displayed.
- ☒ No update occurs.

Resize the browser.

Confirm that:

☒ Edit mode remains responsive.

CHECKPOINT

Before completing this chapter, confirm that:

☒ Customer editing works.

☒ Customer ownership remains protected.

☒ Duplicate detection works.

☒ Dashboard refreshes correctly.

COMMON BEGINNER MISTAKES

Many beginners forget to exclude the customer currently being edited during duplicate detection.

This causes the customer to appear as a duplicate of themselves.

Professional editing workflows always exclude the current record before checking for duplicates.

BEHIND THE SCENES

Customer editing follows the same sequence used during customer creation.

The difference is that the database performs an UPDATE instead of an INSERT.

Every other quality and security check remains the same.

THINK LIKE A SOFTWARE DESIGNER

Consistency makes software easier to understand.

Whenever possible, editing should follow the same rules as creating new records.

WHAT YOU LEARNED

In this lesson you learned how to:

- test customer editing
- verify secure updates
- verify duplicate detection
- protect customer ownership

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now allows authorised users to update customer information securely.

The application reuses the existing customer form, validates updated information, prevents duplicate customer details and refreshes the dashboard automatically after successful updates.

The editing workflow now behaves like that found in many professional CRM systems.

CHAPTER MILESTONE

By the end of Chapter 8, you have successfully built:

- ✓ Customer Edit Mode
- ✓ Secure Customer Retrieval
- ✓ Automatic Form Population
- ✓ Customer Updating
- ✓ Validation During Editing
- ✓ Duplicate Detection During Editing
- ✓ Secure Database Updates
- ✓ Cancel Edit
- ✓ Automatic Dashboard Refresh
- ✓ Customer Editing Security

TRANSITION TO CHAPTER 9

Your Customer Record Management System can now create, display, locate and update customer information.

The final stage of customer management is removing information that is no longer needed.

In the next chapter, you will build secure customer deletion while protecting users from accidentally removing important business information.

CHAPTER 9

DELETING CUSTOMER RECORDS

CHAPTER INTRODUCTION

Creating and updating customer records are important parts of customer management, but businesses also need a safe way to remove records that are no longer required.

Customers may request that their information be deleted.

Duplicate records may need to be removed after careful review.

Some businesses periodically clean their customer database to keep information accurate and relevant.

Deleting information is one of the most sensitive operations in any business application.

Unlike creating or editing a customer, deletion can permanently remove valuable information.

For that reason, professional software always asks for confirmation before removing important records.

In this chapter, you will build a secure customer deletion workflow that protects customer information while giving authorised users complete control over their own records.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Delete Customer action
- Delete confirmation dialog
- Secure customer deletion
- Customer ownership verification
- Dashboard refresh
- Customer Directory refresh

- Dashboard statistics refresh
- Automatic navigation
- Customer deletion testing

LESSON 1

BUILDING SECURE CUSTOMER DELETION

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete customer deletion workflow.

Users will be able to remove customer records directly from the Customer Profile page after confirming that they really want to proceed.

The application will securely delete only customer records belonging to the authenticated user.

After a successful deletion, the Customer Directory and dashboard statistics will update automatically.

WHY THIS MATTERS

Deleting customer records should never happen accidentally.

A single mistaken click could permanently remove valuable business information.

Professional software therefore combines confirmation, authentication and ownership verification before allowing a customer record to be deleted.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Customer editing works.
- ☒ Customer profiles work.
- ☒ Dashboard statistics work.
- ☒ Customer Directory works.
- ☒ DELETE policy exists on the customers table.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Customer Record Management System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `customer-profile.html`
- `customer-profile.js`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Build the complete secure customer deletion workflow.

CUSTOMER-PROFILE.HTML

Keep everything already built.

Inside the Profile Actions section add:

Delete Customer

Use:

`type="button"`

Give the button a clear ID.

Position it separately from:

- Edit Customer
- Back to Dashboard

- Logout

Style it as a destructive action without making it visually distracting.

CUSTOMER-PROFILE.JS

Keep every existing feature working.

Continue supporting:

- Authentication
- Customer profile loading
- Edit Customer
- Call Customer
- Email Customer

DELETE CONFIRMATION

When:

Delete Customer

is selected:

Display a confirmation dialog.

Example:

Are you sure you want to delete this customer?

This action cannot be undone.

If the user cancels:

Stop immediately.

Do not contact Supabase.

DELETE CUSTOMER

If the user confirms:

Delete the customer where:

- id equals the current customer ID
- user_id equals the authenticated user

Continue relying on Row Level Security.

Do not delete another user's customer.

SUCCESS

After a successful deletion:

Display:

Customer deleted successfully.

Automatically return the user to:

dashboard.html

Refresh:

- Customer Directory
- Dashboard statistics

The deleted customer must no longer appear.

ERROR HANDLING

Handle unexpected errors professionally.

Display a friendly message.

Record technical information using:

`console.error()`

Do not expose raw database errors.

DASHBOARD.JS

Keep every existing feature working.

When returning from a successful deletion:

Refresh:

- Customer Directory
- Dashboard statistics

Do not require a manual browser refresh.

AUTH.CSS

Keep every existing style.

Style:

- Delete Customer button
- Hover state
- Keyboard focus
- Confirmation messaging
- Responsive profile actions

Make the Delete Customer button clearly different from Edit Customer without dominating the page.

IMPORTANT

Everything already built must continue working.

Continue relying on Row Level Security.

Delete only authorised customer records.

Do not remove customer records without confirmation.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- customer-profile.html
- customer-profile.js
- dashboard.js
- auth.css

The application should now support secure customer deletion.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open a customer profile.

Select:

Delete Customer

Confirm that:

☒ Confirmation appears.

Cancel the confirmation.

Confirm that:

☒ Customer remains.

Open the confirmation again.

Confirm the deletion.

Verify that:

☒ Customer is deleted.

☒ Dashboard opens.

☒ Customer Directory refreshes.

☒ Dashboard statistics update.

Attempt to delete another user's customer.

Confirm that:

- ☒ Deletion does not occur.
- ☒ Customer information remains protected.

CHECKPOINT

Before moving on, confirm that:

- ☒ Confirmation works.
- ☒ Customer deletion works.
- ☒ Dashboard refreshes.
- ☒ Statistics refresh.
- ☒ Customer ownership remains protected.

COMMON BEGINNER MISTAKES

- Some beginners delete records immediately after the Delete button is selected.
- Professional software always asks the user to confirm destructive actions.
- Another common mistake is deleting records using only the customer ID.
- Every deletion should also verify the authenticated user's ownership.

BEHIND THE SCENES

- Deleting a customer follows the same security principles used throughout this workbook.
- The application confirms the user's intention, verifies authentication, checks customer ownership and then performs the database operation.
- Each layer reduces the risk of accidental or unauthorised deletion.

THINK LIKE A SOFTWARE DESIGNER

- The more serious the action, the more carefully it should be designed.

Users should always understand the consequences before permanently removing important information.

CODE-READING QUESTION

Open `customer-profile.js`. Find one function that supports building secure customer deletion. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build secure customer deletion
- protect customer ownership
- confirm destructive actions
- refresh application data automatically

CHAPTER 9

DELETING CUSTOMER RECORDS

LESSON 2

TESTING CUSTOMER DELETION

Estimated Time

40 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete customer deletion workflow.

You will confirm that authorised users can safely remove customer records while customer privacy and ownership remain protected.

WHY THIS MATTERS

Deleting information is irreversible.

Testing helps ensure that only the intended customer is removed and that the rest of the application continues working correctly afterwards.

BEFORE YOU CONTINUE

Prepare:

- Two authenticated user accounts.
- Several customer records.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Confirm that:

- ☒ Delete Customer appears.
- ☒ Confirmation dialog appears.
- ☒ Cancelling deletion leaves the customer unchanged.
- ☒ Confirming deletion removes the customer.
- ☒ Dashboard statistics update.
- ☒ Customer Directory refreshes.
- ☒ Deleted customer profile no longer opens.

Attempt to delete another user's customer.

Confirm that:

- ☒ Deletion is prevented.
- ☒ Customer information remains protected.

Delete several customers one after another.

Confirm that:

- ☒ Customer Directory remains accurate.
- ☒ Dashboard statistics remain accurate.
- ☒ No unexpected errors occur.

Test the application on a smaller screen.

Confirm that:

- ✓ Delete Customer remains accessible.
- ✓ Confirmation remains understandable.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Customer deletion works.
- ✓ Confirmation works.
- ✓ Dashboard refreshes.
- ✓ Customer ownership remains protected.
- ✓ Dashboard statistics remain accurate.

COMMON BEGINNER MISTAKES

Many beginners test only successful deletion.

Professional testing also confirms that cancelled deletions leave customer information untouched and that another user's records remain protected.

BEHIND THE SCENES

Deleting a customer affects several parts of the application.

Removing the database record is only one step.

The Customer Directory, dashboard statistics and customer profile pages must also respond correctly after the deletion.

THINK LIKE A SOFTWARE DESIGNER

Every destructive action should be tested more thoroughly than ordinary features.

Protecting user information is one of the primary responsibilities of professional software.

WHAT YOU LEARNED

In this lesson you learned how to:

- test secure deletion
- verify ownership protection
- verify dashboard updates
- verify application consistency

CHAPTER SUMMARY

Congratulations.

Your Customer Record Management System now supports the complete customer lifecycle.

Users can create, view, search, update and delete customer records securely.

Every operation respects authentication, customer ownership and Row Level Security.

The application now behaves like a complete customer management solution suitable for many small businesses.

CHAPTER MILESTONE

By the end of Chapter 9, you have successfully built:

- ✓ Secure Customer Deletion
- ✓ Delete Confirmation
- ✓ Protected Customer Ownership
- ✓ Automatic Dashboard Refresh
- ✓ Automatic Customer Directory Refresh
- ✓ Professional Deletion Workflow
- ✓ Customer Deletion Testing

TRANSITION TO CHAPTER 10

Your Customer Record Management System is now functionally complete.

The final chapter focuses on ensuring that every feature works together reliably.

You will perform a complete end-to-end review of the application, verify every business workflow and prepare the software for deployment.

CHAPTER 10

FINAL APPLICATION REVIEW

CHAPTER INTRODUCTION

Congratulations.

You have successfully built a complete Customer Record Management System from the ground up.

Your application now supports the complete customer lifecycle.

Users can:

- Register
- Verify their email address
- Sign in securely
- Create customer records
- View customer records
- Search and organise customer records
- Edit customer information
- Delete customer records

Every feature has been built using HTML, CSS, JavaScript and Supabase without relying on a traditional software development framework.

Before publishing software for real users, professional developers perform a complete system review.

Rather than testing individual features in isolation, they test the entire application from beginning to end.

In this chapter, you will do exactly the same.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Perform complete end-to-end testing
- Verify customer security
- Verify Row Level Security
- Test responsive layouts
- Test application usability
- Prepare the application for deployment

LESSON 1

COMPLETE SYSTEM TESTING

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will perform a complete review of your Customer Record Management System.

Instead of testing one feature at a time, you will test the application exactly as a real business user would.

By the end of this lesson, you should have complete confidence that every feature works correctly before publishing your software.

WHY THIS MATTERS

Software is only valuable when all its individual parts work together reliably.

Professional software development does not end when the last feature is written.

It ends when the complete application has been tested thoroughly.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts
- Several Individual customers
- Several Business customers
- Customers with and without email addresses
- Customers with complete and incomplete optional information

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

If you discover a problem, correct it before deploying your application.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

USER ACCOUNT TESTING

Confirm that:

- ☒ Registration works.
- ☒ Email verification works.
- ☒ Login works.
- ☒ Logout works.
- ☒ Unauthenticated visitors are redirected correctly.

CUSTOMER CREATION

Confirm that:

- ☒ Customer validation works.
- ☒ Customer normalisation works.
- ☒ Duplicate email detection works.
- ☒ Duplicate phone detection works.
- ☒ Customer records save successfully.
- ☒ Dashboard statistics update automatically.

CUSTOMER DIRECTORY

Confirm that:

- ☒ Customer cards display correctly.
- ☒ Customer initials display correctly.
- ☒ Click-to-call works.
- ☒ Click-to-email works.
- ☒ Customer cards refresh automatically.
- ☒ Loading state works.
- ☒ Empty state works.
- ☒ Error state works.

CUSTOMER PROFILE

Confirm that:

- ☒ Customer profiles load correctly.
- ☒ Contact actions work.
- ☒ Edit Customer works.
- ☒ Customer information displays correctly.
- ☒ Dates display correctly.

CUSTOMER DISCOVERY

Confirm that:

- ☒ Search works.
- ☒ Filtering works.
- ☒ Sorting works.
- ☒ Result counter updates.
- ☒ No-results state works.
- ☒ Clear Search and Filters works.

CUSTOMER EDITING

Confirm that:

- ☒ Customer information loads.
- ☒ Validation works.
- ☒ Duplicate detection works.
- ☒ Customer updates successfully.
- ☒ Dashboard refreshes automatically.

CUSTOMER DELETION

Confirm that:

- ☒ Delete confirmation appears.
- ☒ Customer deletes successfully.
- ☒ Dashboard refreshes.
- ☒ Customer Directory refreshes.
- ☒ Statistics refresh correctly.

SECURITY

Confirm that:

- ☒ One user cannot access another user's customer records.
- ☒ One user cannot edit another user's customer.
- ☒ One user cannot delete another user's customer.
- ☒ Row Level Security remains enabled.

RESPONSIVE DESIGN

Reduce the browser width.

Confirm that:

- ☒ Dashboard remains usable.
- ☒ Customer forms remain usable.
- ☒ Customer cards remain readable.
- ☒ Customer profiles remain readable.

- ✓ Toolbar remains usable.
- ✓ Buttons remain easy to select.
- ✓ Horizontal scrolling does not appear unnecessarily.

PERFORMANCE

Create several customer records.

Confirm that:

- ✓ Customer Directory loads quickly.
- ✓ Search remains responsive.
- ✓ Filtering remains responsive.
- ✓ Sorting remains responsive.

CHECKPOINT

Before publishing your software, confirm that:

- ✓ Every feature works.
- ✓ Authentication works.
- ✓ Customer ownership is protected.
- ✓ Dashboard statistics remain accurate.
- ✓ Customer Directory remains accurate.
- ✓ Customer profiles remain accurate.
- ✓ Responsive layouts work correctly.
- ✓ No unexpected JavaScript errors appear.

COMMON BEGINNER MISTAKES

Some developers stop testing after the application appears to work once.

Professional developers continue testing until they are confident that users can complete every important workflow successfully.

Another common mistake is testing only with one user account.

Always test with multiple accounts to confirm that customer privacy remains protected.

BEHIND THE SCENES

Every feature in your Customer Record Management System depends on the others.

Authentication protects customer information.

Customer creation supplies the directory.

The directory opens customer profiles.

Customer editing updates those profiles.

Customer deletion removes them.

Dashboard statistics reflect every change.

Testing confirms that these connected features continue working together.

THINK LIKE A SOFTWARE DESIGNER

Software development is not simply about writing code.

It is about delivering reliable software that people can trust.

Every hour spent testing before deployment saves many hours of fixing problems later.

WHAT YOU LEARNED

In this lesson you learned how to:

- perform complete application testing
- verify customer security
- verify software reliability
- prepare software for deployment

FINAL TESTING

Before considering this workbook complete, perform one final review.

Create a completely new user account.

Begin with an empty customer database.

Work through the application exactly as a first-time customer would.

Complete the following journey:

- ☒ Register
- ☒ Verify email
- ☒ Sign in
- ☒ Create several customers
- ☒ View customer cards
- ☒ Open customer profiles
- ☒ Search customers
- ☒ Filter customers
- ☒ Sort customers
- ☒ Edit one customer
- ☒ Delete one customer
- ☒ Sign out
- ☒ Sign in again

Confirm that every remaining customer record still exists and every dashboard statistic is correct.

If every step succeeds without unexpected errors, your Customer Record Management System is ready to be published.

GOING LIVE

Your software is now ready to be published on the internet.

Publish your complete project to Netlify using the same approach you learned earlier in the Prompt to Profit™ programme.

After deployment, test the live version again.

Do not assume that software will behave exactly the same way after deployment.

Repeat the most important tests on the live application, including:

- Registration
- Login
- Customer creation
- Customer editing
- Customer deletion
- Customer search
- Customer profiles

Only begin sharing your application after confirming that everything works correctly online.

PORTFOLIO DESCRIPTION

Congratulations.

You have successfully built a fully functional Customer Record Management System.

Your application demonstrates practical software development skills including:

- User authentication
- Database design
- Secure customer management
- CRUD operations
- Dashboard design
- Search and filtering
- Responsive layouts
- Database security using Row Level Security

This project is an excellent addition to your software development portfolio because it solves a genuine business problem and demonstrates that you can build secure, data-driven applications using Artificial Intelligence.

REFLECTION QUESTIONS

Before moving to the next workbook, take a few minutes to think about what you have learned.

1.

Which part of this project helped you understand software development better?

2.

How did AI help you build this application more efficiently?

3.

Which software development concepts now make more sense than they did before beginning this workbook?

4.

If you were building this application for a real business, what additional features would you consider adding?

5.

What part of the project challenged you the most, and how did you solve it?

Writing down your answers will help reinforce what you have learned.

EXTENSION CHALLENGES

If you would like to improve your Customer Record Management System further, consider adding some of these features.

- Customer profile photographs
- Customer tags
- Favourite customers
- Customer birthdays
- Customer interaction history
- Customer follow-up reminders
- Customer document uploads
- Customer export to CSV
- Customer import from CSV
- Customer activity log

Do not attempt to build every feature immediately.

Choose one feature at a time and treat it as a separate software project.

NEXT WORKBOOK

Excellent work.

You have completed Workbook 02 of the Prompt to Profit™ Software Workbook Series.

In Workbook 03, you will build a professional Quotation Generator.

Businesses prepare quotations every day for potential customers.

In the next workbook, you will learn how to build software that allows businesses to:

- Create quotations
- Add multiple quotation items
- Perform automatic calculations
- Calculate taxes and discounts
- Save quotations
- Retrieve previous quotations
- Print professional quotation documents
- Manage quotations securely using Supabase

Many of the techniques you learned in this workbook will be reused, while introducing new concepts that expand your software development skills even further.

Congratulations once again on completing your Customer Record Management System.